HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



DATABASE DESIGN TERM PROJECT

Database system for a construction company with multiple projects
(Phase 3: Normalization & Physical database design)

Course: Database [E24-Class 5].
Team: 5.
Members:
Lê Anh Minh - B24DCCE180.
Ứng Trọng Trình - B24DCCE271.
Nguyễn Đinh Anh Quân - B24DCCE222.

Instructor: Nguyễn Đình Hoá.

TABLE OF CONTENTS

1. Introduction & Final Relational Schema 3
2. Functional Dependencies & Normalization 4
2.1 Overview 4
2.2 Detailed Normalization Analysis 4
2.2.1 Subsystem 1: Organization & Human Resources 4
2.2.2 Subsystem 2: Project Management & Work Breakdown Structure (WBS) 6
2.2.3 Subsystem 3: Heavy Machinery & Fleet Maintenance 9
2.2.4 Subsystem 4: Procurement, Materials & Site Inventory 10
2.2.5 Subsystem 5: Trade Subcontractors 13
2.2.6 Subsystem 6: Health, Safety & Environment (HSE) 14
2.3 Master Summary of 3NF Normalized Relational Schema 15
3. Physical Database Design & Implementation Details 19
3.1 DBMS Selection & Configuration 19
3.2 Database Scripts Organization & Proofs. 19
3.2.1 01_schema_ddl.sql (DDL — 29 Core Tables & Indexes): 19
3.2.2 02_triggers_procedures.sql (8 Business Rule Triggers): 19
3.2.3 03_sample_data_dml.sql (Realistic Sample Data): 24
3.2.4 04_business_queries.sql (Operational & Analytical Queries): 26
3.2.5 05_user_queries.sql (User Queries — Multi-Table Joins, Grouping & Filtering) 32
4. Conclusion 34

1. INTRODUCTION & FINAL RELATIONAL SCHEMA

This Phase 3 report completes the **logical normalization** and **physical implementation** of the **A34Build** enterprise database system on **MySQL**, building upon the Conceptual Design (ERD) and Relational Schema Mapping established in Phase 2.

The system covers **6 core operational subsystems** across **29 normalized tables** (plus 1 automated audit log table, EMERGENCY_DISPATCH_LOG, for safety alerting):

1. **Organization & Human Resources (HR):** DEPARTMENT, EMPLOYEE, EMPLOYEE_PHONE, EMPLOYEE_CERTIFICATION, DEPENDENT.
2. **Project Management & WBS:** CLIENT, CLIENT_PHONE, PROJECT, PROJECT_SITE, PROJECT_PHASE, TASK, PRECEDES, WORKS_ON_TASK
3. **Heavy Machinery & Fleet Maintenance:** EQUIPMENT, MAINTENANCE_RECORD, EQUIPMENT_DEPLOYMENT
4. **Procurement, Materials & Site Inventory:** SUPPLIER, SUPPLIER_PHONE, MATERIAL, PURCHASE_ORDER, PO_ITEM, SITE_INVENTORY, DELIVERY_DISPATCH, MATERIAL_USAGE
5. **Trade Subcontractors:** SUBCONTRACTOR, SUBCONTRACT
6. **Health, Safety & Environment (HSE):** SAFETY_INSPECTION, INCIDENT_REPORT, EMERGENCY_DISPATCH_LOG (trigger audit table)

2. FUNCTIONAL DEPENDENCIES & NORMALIZATION

2.1 Overview

All 29 relational schemas mapped from the Phase 2 ERD were audited against 1NF, 2NF, and 3NF criteria (Elmasri & Navathe, *Fundamentals of Database Systems*, 6th ed., Chapter 14):

- **27 schemas** satisfy **3NF** directly without structural modification.
- **2 schemas** exhibited normal form violations from initial ERD mapping and were decomposed/corrected:

Table	Initial Defect (Phase 2)	Violation	3NF Correction (Phase 3)
16. EQUIPMENT_ DEPLOYMENT	SiteID was in the PK: (EquipmentID, SiteID, DeploymentStartDate) , but {EquipmentID,DeploymentStartDate} → SiteID.	**2NF**(Partial Key Dependency)	Moved SiteID out of PK into a non-prime FK. Minimal PK: (EquipmentID, DeploymentStartDate).
24. DELIVERY_ DISPATCH	Contained both PO_ID and SupplierID, where PO_ID → SupplierID via PURCHASE_ORDER.	**3NF** (Transitive Dependency: DeliveryCode → PO_ID → SupplierID)	Removed SupplierID. Vendor is resolved via FK PO_ID → PURCHASE_ORDER.

2.2 Detailed Normalization Analysis

Each table is evaluated using a uniform template: Relational Schema, Functional Dependencies (F), Candidate Keys, Attribute Classification, and Normal Form Verification (1NF → 2NF → 3NF).

2.2.1 Subsystem 1: Organization & Human Resources

1. Table: DEPARTMENT

- Relational Schema: DEPARTMENT(DeptID,DeptName,OfficeLocation,Budget,HeadEmpID)
- Functional Dependencies (F_1):
  - $FD_{1.1}$:DeptID → {DeptName, OfficeLocation, Budget, HeadEmpID} (Department ID uniquely identifies the department).
  - $FD_{1.2}$:DeptName → {DeptID, OfficeLocation, Budget, HeadEmpID} (Department name is unique enterprise-wide).
  - $FD_{1.3}$:HeadEmpID → {DeptID, DeptName, OfficeLocation, Budget} (An employee heads at most one department).
- Candidate Keys & Attributes:
  - Candidate Keys: CK_1 = {DeptID} (PK), CK_2 = {DeptName} (AK), CK_3 = {HeadEmpID} (AK).
  - Prime Attributes: DeptID, DeptName, HeadEmpID.
  - Non-Prime Attributes: OfficeLocation, Budget.
- Normal Form Evaluation:
  - 1NF: Satisfied (atomic domains).
  - 2NF: Satisfied (all candidate keys are single attributes; no partial dependencies possible).

    - 3NF: Satisfied (all determinants are superkeys; no transitive dependencies of non-prime attributes).
- Conclusion: Achieves 3NF.

2. Table: EMPLOYEE

- Relational Schema:
  EMPLOYEE(EmpID,FirstName,MiddleName,LastName,Street,Ward,District,City,DOB,HireDate,Salary, JobTitle,Email,DeptID,SupervisorID)
- Functional Dependencies (F_2):
    - $FD_{2.1}$:EmpID → {FirstName, MiddleName, LastName, Street, Ward, District, City, DOB, HireDate, Salary, JobTitle, Email, DeptID, SupervisorID}
    - (Employee ID uniquely identifies the employee).
    - $FD_{2.2}$:Email → {EmpID, FirstName, MiddleName, LastName, Street, Ward, District, City, DOB, HireDate, Salary, JobTitle, DeptID, SupervisorID} (Corporate email is unique).
- Candidate Keys & Attributes:
    - Candidate Keys: $CK_1 = \{\text{EmpID}\}$ (PK), $CK_2 = \{\text{Email}\}$ (AK).
    - Prime Attributes: EmpID, Email.
    - Non-Prime Attributes: FirstName, MiddleName, LastName, Street, Ward, District, City, DOB, HireDate, Salary, JobTitle, DeptID, SupervisorID.
- Normal Form Evaluation:
    - 1NF: Satisfied (composite name and address attributes decomposed into atomic columns; multivalued attributes separated).
    - 2NF: Satisfied (candidate keys are single attributes; no partial dependencies).
    - 3NF: Satisfied (all determinants are superkeys; no transitive dependencies).
- Conclusion: Achieves 3NF.

3. Table: EMPLOYEE_PHONE

- Relational Schema: EMPLOYEE_PHONE(EmpID, PhoneNumber)
- Functional Dependencies (F_3): Trivial dependencies only ({EmpID, PhoneNumber} → {EmpID, PhoneNumber}).
- Candidate Keys & Attributes:
    - Candidate Key: $CK = \{\text{EmpID, PhoneNumber}\}$ (PK).
    - Prime Attributes: EmpID, PhoneNumber.
    - Non-Prime Attributes: $\emptyset$ (None).
- Normal Form Evaluation:
    - 1NF: Satisfied (single atomic phone number per tuple).
    - 2NF / 3NF: Trivially satisfied (all attributes are prime).
- Conclusion: Achieves 3NF.

4. Table: EMPLOYEE_CERTIFICATION

- Relational Schema: EMPLOYEE_CERTIFICATION(EmpID, Certification)
- Functional Dependencies (F_4): Trivial dependencies only.
- Candidate Keys & Attributes:
    - Candidate Key: $CK = \{\text{EmpID, Certification}\}$ (PK).
    - Prime Attributes: EmpID, Certification.
    - Non-Prime Attributes: $\emptyset$ (None).

- Normal Form Evaluation:
  - 1NF / 2NF / 3NF: Trivially satisfied (all attributes are prime).
- Conclusion: Achieves 3NF.

5. Table: DEPENDENT

- Relational Schema:
  DEPENDENT(EmpID, DependentName,Gender,BirthDate,Relationship,ContactPhone,IsEmergencyContact)
- Functional Dependencies (F_5):
  - $FD_{5.1}$: {EmpID, DependentName} → {Gender, BirthDate, Relationship, ContactPhone, IsEmergencyContact} (Weak entity identified by employee and partial key).
- Candidate Keys & Attributes:
  - Candidate Key: $CK = \{\text{EmpID, DependentName}\}$ (PK).
  - Prime Attributes: EmpID, DependentName.
  - Non-Prime Attributes: Gender, BirthDate, Relationship, ContactPhone, IsEmergencyContact.
- Normal Form Evaluation:
  - 1NF: Satisfied (atomic domains).
  - 2NF: Satisfied (neither EmpID alone nor DependentName alone determines dependent attributes; no partial dependency).
  - 3NF: Satisfied (determinant is the candidate key; no transitive dependencies).
- Conclusion: Achieves 3NF.

2.2.2 Subsystem 2: Project Management & Work Breakdown Structure (WBS)

6. Table: CLIENT

- Relational Schema: CLIENT(ClientID,ClientName,ClientType,ContactPerson,ContactEmail,TaxCode)
- Functional Dependencies (F_6):
  - $FD_{6.1}$:ClientID → {ClientName, ClientType, ContactPerson, ContactEmail, TaxCode}
  - $FD_{6.2}$:TaxCode → {ClientID, ClientName, ClientType, ContactPerson, ContactEmail} (Corporate tax code is unique).
- Candidate Keys & Attributes:
  - Candidate Keys: $CK_1 = \{\text{ClientID}\}$ (PK), $CK_2 = \{\text{TaxCode}\}$ (AK).
  - Prime Attributes: ClientID, TaxCode.
  - Non-Prime Attributes: ClientName, ClientType, ContactPerson, ContactEmail.
- Normal Form Evaluation:
  - 1NF / 2NF: Satisfied (single-attribute keys; no partial dependencies).
  - 3NF: Satisfied (all determinants are superkeys).
- Conclusion: Achieves 3NF.

7. Table: CLIENT_PHONE

- Relational Schema: CLIENT_PHONE(ClientID, ContactPhone)
- Functional Dependencies (F_7): Trivial dependencies only.
- Candidate Keys & Attributes:
  - Candidate Key: $CK = \{\text{ClientID, ContactPhone}\}$ (PK).
  - Prime Attributes: ClientID, ContactPhone.

- Non-Prime Attributes: ∅ (None).
- Normal Form Evaluation:
  - 1NF / 2NF / 3NF: Trivially satisfied (all attributes are prime).
- Conclusion: Achieves 3NF.

8. Table: PROJECT

- Relational Schema:
  PROJECT(ProjectID,ProjectName,StartDate,PlannedEndDate,ActualEndDate,EstimatedBudget,Status,
  ClientID,DeptID,DirectorEmpID)
- Functional Dependencies (F_8):
  - $FD_{8.1}$:ProjectID →
    {ProjectName, StartDate, PlannedEndDate, ActualEndDate, EstimatedBudget, Status, ClientID, DeptID, DirectorEmpID}
  - $FD_{8.2}$:ProjectName →
    {ProjectID, StartDate, PlannedEndDate, ActualEndDate, EstimatedBudget, Status, ClientID, DeptID, DirectorEmpID}
    (Contract title is unique).
- Candidate Keys & Attributes:
  - Candidate Keys: $CK_1 = \{\text{ProjectID}\}$ (PK), $CK_2 = \{\text{ProjectName}\}$ (AK).
  - Prime Attributes: ProjectID, ProjectName.
  - Non-Prime Attributes: StartDate, PlannedEndDate, ActualEndDate, EstimatedBudget, Status, ClientID, DeptID, DirectorEmpID.
- Normal Form Evaluation:
  - 1NF / 2NF: Satisfied (single-attribute keys; no partial dependencies).
  - 3NF: Satisfied (all determinants are superkeys; foreign keys reference external entities without creating intra-table transitive dependencies).
- Conclusion: Achieves 3NF.

9. Table: PROJECT_SITE

- Relational Schema:
  PROJECT_SITE(SiteID,ProjectID,SiteName,Latitude,Longitude,StreetAddress,District,City_Province,
- EnvironmentalPermit,SiteStatus)
- Functional Dependencies (F_9):
  - $FD_{9.1}$:SiteID → {ProjectID, SiteName, Latitude, Longitude, StreetAddress, District, City_Province, EnvironmentalPermit, SiteStatus}
  - $FD_{9.2}$: {ProjectID, SiteName} →
    {SiteID, Latitude, Longitude, StreetAddress, District, City_Province, EnvironmentalPermit, SiteStatus}
    (Site name is unique per project).
  - $FD_{9.3}$: {Latitude, Longitude} →
    {SiteID, ProjectID, SiteName, StreetAddress, District, City_Province, EnvironmentalPermit, SiteStatus}
    (Geographic coordinates identify a site).
- Candidate Keys & Attributes:
  - Candidate Keys: $CK_1 = \{\text{SiteID}\}$ (PK), $CK_2 = \{\text{ProjectID, SiteName}\}$ (AK), $CK_3 =$ {Latitude, Longitude} (AK).
  - Prime Attributes: SiteID, ProjectID, SiteName, Latitude, Longitude.
  - Non-Prime Attributes: StreetAddress, District, City_Province, EnvironmentalPermit, SiteStatus.
- Normal Form Evaluation:
  - 1NF: Satisfied (atomic fields).

    - 2NF: Satisfied (for composite keys CK_2 and CK_3, no proper subset determines any non-prime attribute).
    - 3NF: Satisfied (all determinants are superkeys).
- Conclusion: Achieves 3NF.

10. Table: PROJECT_PHASE

- Relational Schema: PROJECT_PHASE(ProjectID, PhaseNumber,PhaseTitle,StartDate,TargetCompletionDate,PhaseBudget,PhaseStatus)
- Functional Dependencies (F_{10}):
    - $FD_{10.1}$: {ProjectID, PhaseNumber} → {PhaseTitle, StartDate, TargetCompletionDate, PhaseBudget, PhaseStatus}
- Candidate Keys & Attributes:
    - Candidate Key: $CK = $ {ProjectID, PhaseNumber} (PK).
    - Prime Attributes: ProjectID, PhaseNumber.
    - Non-Prime Attributes: PhaseTitle, StartDate, TargetCompletionDate, PhaseBudget, PhaseStatus.
- Normal Form Evaluation:
    - 1NF: Satisfied.
    - 2NF: Satisfied (full functional dependency on composite key; neither ProjectID nor PhaseNumber alone determines phase attributes).
    - 3NF: Satisfied (determinant is the candidate key).
- Conclusion: Achieves 3NF.

11. Table: TASK

- Relational Schema: TASK(TaskID,TaskName,EstimatedHours,ScheduledStartDate,ScheduledEndDate,ActualEndDate, PriorityLevel,TaskStatus,ProjectID,PhaseNumber)
- Functional Dependencies (F_{11}):
    - $FD_{11.1}$:TaskID → {TaskName, EstimatedHours, ScheduledStartDate, ScheduledEndDate, ActualEndDate, PriorityLevel, TaskStatus, ProjectID, PhaseNumber}
    - $FD_{11.2}$: {ProjectID, PhaseNumber, TaskName} → {TaskID, EstimatedHours, ScheduledStartDate, ScheduledEndDate, ActualEndDate, PriorityLevel, TaskStatus}
- Candidate Keys & Attributes:
    - Candidate Keys: $CK_1 = $ {TaskID} (PK), $CK_2 = $ {ProjectID, PhaseNumber, TaskName} (AK).
    - Prime Attributes: TaskID, ProjectID, PhaseNumber, TaskName.
    - Non-Prime Attributes: EstimatedHours, ScheduledStartDate, ScheduledEndDate, ActualEndDate, PriorityLevel, TaskStatus.
- Normal Form Evaluation:
    - 1NF / 2NF: Satisfied (no partial dependencies on either candidate key).
    - 3NF: Satisfied (all determinants are superkeys; composite foreign key (ProjectID, PhaseNumber) does not determine other non-prime attributes).
- Conclusion: Achieves 3NF.

12. Table: PRECEDES

- Relational Schema: PRECEDES(PredecessorTaskID, SuccessorTaskID)
- Functional Dependencies (F_{12}): Trivial dependencies only.

- Candidate Keys & Attributes:
  - Candidate Key: $CK = \{\text{PredecessorTaskID, SuccessorTaskID}\}$ (PK).
  - Prime Attributes: PredecessorTaskID, SuccessorTaskID.
  - Non-Prime Attributes: $\emptyset$ (None).
- Normal Form Evaluation:
  - 1NF / 2NF / 3NF: Trivially satisfied (all attributes are prime).
- Conclusion: Achieves 3NF.

13. Table: WORKS_ON_TASK

- Relational Schema: WORKS_ON_TASK(EmpID, TaskID,AssignedDate,RoleOnTask,HoursLogged)
- Functional Dependencies (F_{13}):
  - $FD_{13.1}$:{EmpID, TaskID} → {AssignedDate, RoleOnTask, HoursLogged}
- Candidate Keys & Attributes:
  - Candidate Key: $CK = \{\text{EmpID, TaskID}\}$ (PK).
  - Prime Attributes: EmpID, TaskID.
  - Non-Prime Attributes: AssignedDate, RoleOnTask, HoursLogged.
- Normal Form Evaluation:
  - 1NF: Satisfied.
  - 2NF: Satisfied (full functional dependency; neither EmpID nor TaskID alone determines logged hours).
  - 3NF: Satisfied (determinant is the candidate key).

Conclusion: Achieves 3NF.

2.2.3 Subsystem 3: Heavy Machinery & Fleet Maintenance

14. Table: EQUIPMENT

- Relational Schema: EQUIPMENT(EquipmentID,Model,SerialNumber,Category,Ownership,HourlyRate,CurrentStatus)
- Functional Dependencies (F_{14}):
  - $FD_{14.1}$:EquipmentID → {Model, SerialNumber, Category, Ownership, HourlyRate, CurrentStatus}
  - $FD_{14.2}$:SerialNumber → {EquipmentID, Model, Category, Ownership, HourlyRate, CurrentStatus} (Manufacturer chassis/VIN is unique).
- Candidate Keys & Attributes:
  - Candidate Keys: $CK_1 = \{\text{EquipmentID}\}$ (PK), $CK_2 = \{\text{SerialNumber}\}$ (AK).
  - Prime Attributes: EquipmentID, SerialNumber.
  - Non-Prime Attributes: Model, Category, Ownership, HourlyRate, CurrentStatus.
- Normal Form Evaluation:
  - 1NF / 2NF: Satisfied (single-attribute keys; no partial dependencies).
  - 3NF: Satisfied (all determinants are superkeys).
- Conclusion: Achieves 3NF.

15. Table: MAINTENANCE_RECORD

- Relational Schema: MAINTENANCE_RECORD(EquipmentID, RecordNumber,ServiceDate,MaintenanceType,Cost,ServiceVendor)
- Functional Dependencies (F_{15}):
  - $FD_{15.1}$:{EquipmentID, RecordNumber} → {ServiceDate, MaintenanceType, Cost, ServiceVendor}
- Candidate Keys & Attributes:

    - Candidate Key: $CK = \{\text{EquipmentID, RecordNumber}\}$ (PK).
    - Prime Attributes: EquipmentID, RecordNumber.
    - Non-Prime Attributes: ServiceDate, MaintenanceType, Cost, ServiceVendor.
- Normal Form Evaluation:
    - 1NF / 2NF: Satisfied (full functional dependency on composite key).
    - 3NF: Satisfied (determinant is the candidate key).
- Conclusion: Achieves 3NF.

16. Table: EQUIPMENT_DEPLOYMENT (2NF Partial Key Resolution)

1. Phase 2 Initial Design & Conflict Analysis:

- **Initial Relational Schema:**
  EQUIPMENT_DEPLOYMENT(EquipmentID, SiteID, DeploymentStartDate,DeploymentEndDate, HoursOperated,OperatorEmpID)
- **Initial Key & Functional Dependencies:**
    - Initial Composite Key: {EquipmentID, SiteID, DeploymentStartDate}
    - Logistics Dependency ($FD_{16.1}$): {EquipmentID, DeploymentStartDate} → {SiteID, DeploymentEndDate, HoursOperated, OperatorEmpID} *(A machine operates at exactly one site per start date)*.
- **Conflict & 2NF Violation:**
    - **Violation Analysis:** In $FD_{16.1}$, the attribute SiteID is functionally determined by {EquipmentID, DeploymentStartDate}, which is a **proper subset** of the initial 3-part key.
    - **Conflict:** This constitutes a **Partial Key Dependency**, directly violating **2NF**. The initial key is non-minimal and redundant.

2. Conflict Resolution & Normalized Design (3NF Certified):

- **Resolution Strategy:** Remove SiteID from the composite primary key and retain it as a non-prime foreign key referencing PROJECT_SITE. The primary key is minimized to strictly {EquipmentID,DeploymentStartDate}, eliminating the partial key dependency and restoring 2NF/3NF compliance without losing the job site relationship.
- **Normalized Relational Schema:**
  EQUIPMENT_DEPLOYMENT(EquipmentID, DeploymentStartDate,SiteID,DeploymentEndDate, HoursOperated,OperatorEmpID)
- **Normalized Candidate Keys & Attributes:**
    - **Candidate Key:** $CK = \{\text{EquipmentID, DeploymentStartDate}\}$ (Minimal PK).
    - **Prime Attributes:** EquipmentID, DeploymentStartDate.
    - **Non-Prime Attributes:** SiteID, DeploymentEndDate, HoursOperated, OperatorEmpID.
- **Normal Form Evaluation:**
    - **1NF / 2NF:** Satisfied (all non-prime attributes depend fully on the complete 2-attribute minimal key; partial dependency eliminated).
    - **3NF:** Satisfied (the sole determinant is a superkey; no transitive dependencies).
- **Conclusion: Achieves 3NF**.

2.2.4 Subsystem 4: Procurement, Materials & Site Inventory

17. Table: SUPPLIER

- **Relational Schema:** SUPPLIER(SupplierID,CompanyName,TaxCode,ContactEmail,CreditRating)
- **Functional Dependencies (F_{17}):**
    - $FD_{17.1}$:SupplierID → {CompanyName, TaxCode, ContactEmail, CreditRating}
    - $FD_{17.2}$:TaxCode → {SupplierID, CompanyName, ContactEmail, CreditRating} (Tax code is unique).
- **Candidate Keys & Attributes:**

    - **Candidate Keys:** $CK_1 = \{\text{SupplierID}\}$ (PK), $CK_2 = \{\text{TaxCode}\}$ (AK).
    - **Prime Attributes:** SupplierID, TaxCode.
    - **Non-Prime Attributes:** CompanyName, ContactEmail, CreditRating.
- **Normal Form Evaluation:**
    - **1NF / 2NF / 3NF:** Satisfied (single-attribute keys; determinants are superkeys).
- **Conclusion: Achieves 3NF**.

18. Table: SUPPLIER_PHONE

- **Relational Schema:** SUPPLIER_PHONE(SupplierID, ContactPhone)
- **Functional Dependencies (F_{18}):** Trivial dependencies only.
- **Candidate Keys & Attributes:**
    - **Candidate Key:** $CK = \{\text{SupplierID, ContactPhone}\}$ (PK).
    - **Prime Attributes:** SupplierID, ContactPhone.
    - **Non-Prime Attributes:** $\emptyset$ (None).
- **Normal Form Evaluation:**
    - **1NF / 2NF / 3NF:** Trivially satisfied (all attributes are prime).
- **Conclusion: Achieves 3NF**.

19. Table: SUPPLIER_CATEGORY

- **Relational Schema:** SUPPLIER_CATEGORY(SupplierID, SupplyCategory)
- **Functional Dependencies (F_{19}):** Trivial dependencies only.
- **Candidate Keys & Attributes:**
    - **Candidate Key:** $CK = \{\text{SupplierID, SupplyCategory}\}$ (PK).
    - **Prime Attributes:** SupplierID, SupplyCategory.
    - **Non-Prime Attributes:** $\emptyset$ (None).
- **Normal Form Evaluation:**
    - **1NF / 2NF / 3NF:** Trivially satisfied (all attributes are prime).
- **Conclusion: Achieves 3NF**.

20. Table: MATERIAL

- **Relational Schema:** MATERIAL(MaterialID,MaterialName,Category,StandardUnit,UnitPrice,MinStock)
- **Functional Dependencies (F_{20}):**
    - $FD_{20.1}$:MaterialID → {MaterialName, Category, StandardUnit, UnitPrice, MinStock}
    - $FD_{20.2}$:MaterialName → {MaterialID, Category, StandardUnit, UnitPrice, MinStock} (Standard catalog name is unique).
- **Candidate Keys & Attributes:**
    - **Candidate Keys:** $CK_1 = \{\text{MaterialID}\}$ (PK), $CK_2 = \{\text{MaterialName}\}$ (AK).
    - **Prime Attributes:** MaterialID, MaterialName.
    - **Non-Prime Attributes:** Category, StandardUnit, UnitPrice, MinStock.
- **Normal Form Evaluation:**
    - **1NF / 2NF / 3NF:** Satisfied (single-attribute keys; determinants are superkeys).
- **Conclusion: Achieves 3NF**.

21. Table: PURCHASE_ORDER

- **Relational Schema:** PURCHASE_ORDER(PO_ID,OrderDate,ExpectedDeliveryDate,DeliveryStatus,ProjectID,SupplierID)

- **Functional Dependencies (F_{21}):**
  - $FD_{21.1}$:PO_ID → {OrderDate, ExpectedDeliveryDate, DeliveryStatus, ProjectID, SupplierID}
- **Candidate Keys & Attributes:**
  - **Candidate Key:** $CK = \{PO_ID\}$ (PK).
  - **Prime Attributes:** PO_ID.
  - **Non-Prime Attributes:** OrderDate, ExpectedDeliveryDate, DeliveryStatus, ProjectID, SupplierID.
- **Normal Form Evaluation:**
  - **1NF / 2NF / 3NF:** Satisfied (single-attribute key; determinant is a superkey).
- **Conclusion: Achieves 3NF**.

22. Table: PO_ITEM

- **Relational Schema:** PO_ITEM(PO_ID, LineNumber,QuantityOrdered,AgreedUnitPrice,MaterialID)
- **Functional Dependencies (F_{22}):**
  - $FD_{22.1}$: {PO_ID, LineNumber} → {QuantityOrdered, AgreedUnitPrice, MaterialID}
  - $FD_{22.2}$: {PO_ID, MaterialID} → {LineNumber, QuantityOrdered, AgreedUnitPrice} (A material appears once per purchase order).
- **Candidate Keys & Attributes:**
  - **Candidate Keys:** $CK_1 = \{PO_ID, LineNumber\}$ (PK), $CK_2 = \{PO_ID, MaterialID\}$ (AK).
  - **Prime Attributes:** PO_ID, LineNumber, MaterialID.
  - **Non-Prime Attributes:** QuantityOrdered, AgreedUnitPrice.
- **Normal Form Evaluation:**
  - **1NF:** Satisfied.
  - **2NF:** Satisfied (full functional dependency; neither PO_ID nor LineNumber alone determines quantity or price).
  - **3NF:** Satisfied (determinants are candidate keys. AgreedUnitPrice is contract-locked and does not depend on MaterialID, precluding transitive dependency).
- **Conclusion: Achieves 3NF**.

23. Table: SITE_INVENTORY

- **Relational Schema:** SITE_INVENTORY(SiteID, MaterialID,CurrentStockQuantity,SafetyReorderQuantity,LastStocktakeDate)
- **Functional Dependencies (F_{23}):**
  - $FD_{23.1}$: {SiteID, MaterialID} → {CurrentStockQuantity, SafetyReorderQuantity, LastStocktakeDate}
- **Candidate Keys & Attributes:**
  - **Candidate Key:** $CK = \{SiteID, MaterialID\}$ (PK).
  - **Prime Attributes:** SiteID, MaterialID.
  - **Non-Prime Attributes:** CurrentStockQuantity, SafetyReorderQuantity, LastStocktakeDate.
- **Normal Form Evaluation:**
  - **1NF / 2NF / 3NF:** Satisfied (full functional dependency; determinant is candidate key).
- **Conclusion: Achieves 3NF**.

24. Table: DELIVERY_DISPATCH (3NF Transitive Dependency Resolution)

1. Phase 2 Initial Design & Conflict Analysis:

- **Initial Relational Schema:** DELIVERY_DISPATCH(DeliveryCode,SupplierID,MaterialID,SiteID,DeliveryDate,DeliveredQuantity, ReceivedByEmpID,PO_ID)
- **Initial Key & Functional Dependencies:**

- Initial Candidate Key: {DeliveryCode}
- $FD_{24.1}$:DeliveryCode → {SupplierID, MaterialID, SiteID, DeliveryDate, DeliveredQuantity, ReceivedByEmpID, PO_ID}
- $FD_{24.2}$:PO_ID → SupplierID *(Each purchase order is issued to exactly one supplier in PURCHASE_ORDER)*.

- **Conflict & 3NF Violation:**
  - **Violation Analysis:** In $FD_{24.2}$, PO_ID is **NOT a superkey** of DELIVERY_DISPATCH (a purchase order may be fulfilled across multiple delivery batches), and SupplierID is a **non-prime attribute**.
  - **Conflict:** This induces a **Transitive Dependency**:

    DeliveryCode → PO_ID → SupplierID

    violating **3NF**.
  - **Business Anomalies:** Data redundancy (vendor ID duplicated across PO and delivery records); update anomaly (reassigning a PO vendor creates inconsistency in dispatch history).

2. Conflict Resolution & Normalized Design (3NF Certified):

- **Resolution Strategy:** Eliminate SupplierID from DELIVERY_DISPATCH. Because PO_ID → SupplierID is fully maintained in PURCHASE_ORDER, vendor information is retrieved losslessly by joining on the foreign key PO_ID, completely resolving the transitive dependency and eliminating update anomalies.
- **Normalized Relational Schema:**
  DELIVERY_DISPATCH(DeliveryCode,MaterialID,SiteID,DeliveryDate,DeliveredQuantity, ReceivedByEmpID,PO_ID)
- **Normalized Candidate Keys & Attributes:**
  - **Candidate Key:** $CK = \{\text{DeliveryCode}\}$ (PK).
  - **Prime Attributes:** DeliveryCode.
  - **Non-Prime Attributes:** MaterialID, SiteID, DeliveryDate, DeliveredQuantity, ReceivedByEmpID, PO_ID.
- **Normal Form Evaluation:**
  - **1NF / 2NF:** Satisfied (single-attribute candidate key).
  - **3NF:** Satisfied (projecting out SupplierID eliminates the transitive dependency; supplier identity is preserved without loss via foreign key PO_ID → PURCHASE_ORDER).
- **Conclusion: Achieves 3NF**.

25. Table: MATERIAL_USAGE

- **Relational Schema:** MATERIAL_USAGE(TaskID, MaterialID, SiteID, UsageDate,QuantityUsed)
- **Functional Dependencies (F_{25}):**
  - $FD_{25.1}$: {TaskID, MaterialID, SiteID, UsageDate} → {QuantityUsed}
- **Candidate Keys & Attributes:**
  - **Candidate Key:** $CK = \{\text{TaskID, MaterialID, SiteID, UsageDate}\}$ (PK).
  - **Prime Attributes:** TaskID, MaterialID, SiteID, UsageDate.
  - **Non-Prime Attributes:** QuantityUsed.
- **Normal Form Evaluation:**
  - **1NF / 2NF / 3NF:** Satisfied (full functional dependency on 4-part key; determinant is candidate key).
- **Conclusion: Achieves 3NF**.

2.2.5 Subsystem 5: Trade Subcontractors

26. Table: SUBCONTRACTOR

- **Relational Schema:**
  SUBCONTRACTOR(SubcontractorID,CompanyName,TaxCode,ContactEmail,ContactPhone,

TradeSpecialty,LicenseNumber,SafetyRatingScore)

- **Functional Dependencies (F_{26}):**
  - $FD_{26.1}$:SubcontractorID → {CompanyName, TaxCode, ContactEmail, ContactPhone, TradeSpecialty, LicenseNumber, SafetyRatingScore}
  - $FD_{26.2}$:TaxCode → {SubcontractorID, CompanyName, ContactEmail, ContactPhone, TradeSpecialty, LicenseNumber, SafetyRatingScore}
  - $FD_{26.3}$:LicenseNumber → {SubcontractorID, CompanyName, TaxCode, ContactEmail, ContactPhone, TradeSpecialty, SafetyRatingScore} (Trade license is unique).
- **Candidate Keys & Attributes:**
  - **Candidate Keys:** $CK_1 = \{\text{SubcontractorID}\}$ (PK), $CK_2 = \{\text{TaxCode}\}$ (AK), $CK_3 = \{\text{LicenseNumber}\}$ (AK).
  - **Prime Attributes:** SubcontractorID, TaxCode, LicenseNumber.
  - **Non-Prime Attributes:** CompanyName, ContactEmail, ContactPhone, TradeSpecialty, SafetyRatingScore.
- **Normal Form Evaluation:**
  - **1NF / 2NF / 3NF:** Satisfied (all determinants are superkeys).
- **Conclusion: Achieves 3NF**.

27. Table: SUBCONTRACT

- **Relational Schema:** SUBCONTRACT(ContractAgreementNo,ProjectID,SubcontractorID,ScopeDescription,ContractValue, WarrantyHoldRate,StartDate,CompletionDate)
- **Functional Dependencies (F_{27}):**
  - $FD_{27.1}$:ContractAgreementNo → {ProjectID, SubcontractorID, ScopeDescription, ContractValue, WarrantyHoldRate, StartDate, CompletionDate}
- **Candidate Keys & Attributes:**
  - **Candidate Key:** $CK = \{\text{ContractAgreementNo}\}$ (PK).
  - **Prime Attributes:** ContractAgreementNo.
  - **Non-Prime Attributes:** ProjectID, SubcontractorID, ScopeDescription, ContractValue, WarrantyHoldRate, StartDate, CompletionDate.
- **Normal Form Evaluation:**
  - **1NF / 2NF / 3NF:** Satisfied (single-attribute key; determinant is superkey).
- **Conclusion: Achieves 3NF**.

2.2.6 Subsystem 6: Health, Safety & Environment (HSE)

28. Table: SAFETY_INSPECTION

- **Relational Schema:** SAFETY_INSPECTION(InspectionID,InspectionDate,InspectionType,InspectionScope, OverallScore,InspectionResult,ViolationNoted,FixDeadline,SiteID,InspectorEmpID)
- **Functional Dependencies (F_{28}):**
  - $FD_{28.1}$:InspectionID → {InspectionDate, InspectionType, InspectionScope, OverallScore, InspectionResult, ViolationNoted, FixDeadline, SiteID, InspectorEmpID}
- **Candidate Keys & Attributes:**
  - **Candidate Key:** $CK = \{\text{InspectionID}\}$ (PK).

- **Prime Attributes:** InspectionID.
- **Non-Prime Attributes:** InspectionDate, InspectionType, InspectionScope, OverallScore, InspectionResult, ViolationNoted, FixDeadline, SiteID, InspectorEmpID.

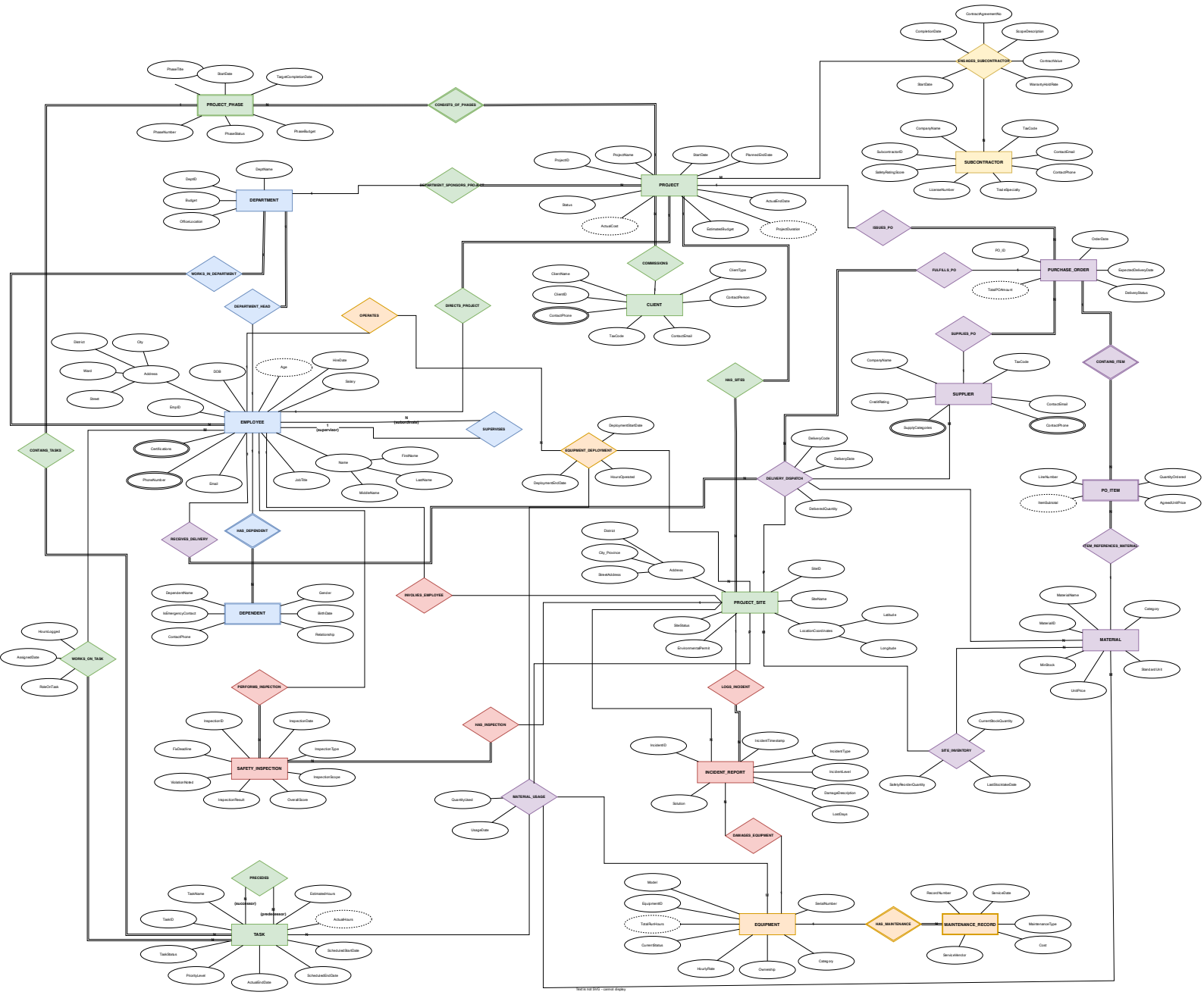
- **Normal Form Evaluation:**
  - **1NF / 2NF / 3NF:** Satisfied (single-attribute key; determinant is superkey).
- **Conclusion: Achieves 3NF**.

29. Table: INCIDENT_REPORT

- **Relational Schema:** INCIDENT_REPORT(IncidentID,IncidentTimestamp,IncidentType,IncidentLevel, DamageDescription,LostDays,Solution,SiteID,InvolvedEmpID,DamagedEquipmentID)
- **Functional Dependencies (F_{29}):**
  - $FD_{29.1}$:IncidentID → {IncidentTimestamp, IncidentType, IncidentLevel, DamageDescription, LostDays, Solution, SiteID, InvolvedEmpID, DamagedEquipmentID}
- **Candidate Keys & Attributes:**
  - **Candidate Key:** $CK = \{\text{IncidentID}\}$ (PK).
  - **Prime Attributes:** IncidentID.
  - **Non-Prime Attributes:** IncidentTimestamp, IncidentType, IncidentLevel, DamageDescription, LostDays, Solution, SiteID, InvolvedEmpID, DamagedEquipmentID.
- **Normal Form Evaluation:**
  - **1NF / 2NF / 3NF:** Satisfied (single-attribute key; determinant is superkey).
- **Conclusion: Achieves 3NF**.

2.3 Master Summary of 3NF Normalized Relational Schema

The following master reference table summarizes the normalization results, candidate keys, and foreign keys across all 29 normalized relations:


PROJECT_PHASE
PhaseTitle
StartDate
TargetCompletionDate
PhaseNumber
PhaseStatus
PhaseBudget
CONSISTS_OF_PHASES
DEPARTMENT
DeptID
DeptName
Budget
OfficeLocation
DEPARTMENT_SPONSORS_PROJECT
PROJECT
ProjectID
ProjectName
StartDate
PlannedEndDate
Status
ActualCost
ActualEndDate
EstimatedBudget
ProjectDuration
ENGAGES_SUBCONTRACTOR
ContractAgreementNo
CompletionDate
ScopeDescription
ContractValue
StartDate
WarrantyHoldRate
SUBCONTRACTOR
CompanyName
TaxCode
SubcontractorID
ContactEmail
SafetyRatingScore
ContactPhone
LicenseNumber
TradeSpecialty
WORKS_IN_DEPARTMENT
DEPARTMENT_HEAD
OPERATES
DIRECTS_PROJECT
COMMISSIONS
CLIENT
ClientName
ClientID
ContactPhone
ClientType
ContactPerson
TaxCode
ContactEmail
ISSUES_PO
FULFILLS_PO
PURCHASE_ORDER
PO_ID
OrderDate
TotalPOAmount
ExpectedDeliveryDate
DeliveryStatus
SUPPLIES_PO
SUPPLIER
CompanyName
TaxCode
CreditRating
ContactEmail
ContactPhone
SupplyCategories
CONTAINS_ITEM
PO_ITEM
LineNumber
QuantityOrdered
ItemSubtotal
AgreedUnitPrice
ITEM_REFERENCES_MATERIAL
MATERIAL
MaterialName
Category
MaterialID
MinStock
StandardUnit
UnitPrice
EMPLOYEE
EmpID
Name
FirstName
MiddleName
LastName
DOB
Age
HireDate
Salary
JobTitle
Email
PhoneNumber
Certifications
Address
District
City
Ward
Street
SUPERVISES
(supervisor)
(subordinate)
HAS_SITES
EQUIPMENT_DEPLOYMENT
DeploymentStartDate
DeploymentEndDate
HoursOperated
DELIVERY_DISPATCH
DeliveryCode
DeliveryDate
DeliveredQuantity
CONTAINS_TASKS
RECEIVES_DELIVERY
HAS_DEPENDENT
DEPENDENT
DependentName
IsEmergencyContact
ContactPhone
Gender
BirthDate
Relationship
INVOLVES_EMPLOYEE
PROJECT_SITE
SiteID
SiteName
SiteStatus
EnvironmentalPermit
LocationCoordinates
Latitude
Longitude
Address
District
City_Province
StreetAddress
WORKS_ON_TASK
HoursLogged
AssignedDate
RoleOnTask
PERFORMS_INSPECTION
SAFETY_INSPECTION
InspectionID
InspectionDate
FixDeadline
InspectionType
ViolationNoted
InspectionScope
InspectionResult
OverallScore
HAS_INSPECTION
LOGS_INCIDENT
INCIDENT_REPORT
IncidentID
IncidentTimestamp
IncidentType
IncidentLevel
DamageDescription
LostDays
Solution
SITE_INVENTORY
CurrentStockQuantity
SafetyReorderQuantity
LastStockUpdateDate
MATERIAL_USAGE
QuantityUsed
UsageDate
DAMAGES_EQUIPMENT
PRECEDES
(successor)
(predecessor)
TASK
TaskID
TaskName
EstimatedHours
ActualHours
TaskStatus
PriorityLevel
ScheduledStartDate
ActualEndDate
ScheduledEndDate
EQUIPMENT
EquipmentID
Model
SerialNumber
TotalRuntimeHours
CurrentStatus
HourlyRate
Ownership
Category
HAS_MAINTENANCE
MAINTENANCE_RECORD
RecordNumber
ServiceDate
MaintenanceType
Cost
ServiceVendor
Next is full SVG – cannot display

No.	Relational Schema Name	Primary Key (PK)	Alternate Candidate Keys (AK)	Foreign Keys (FK) & Referencing Targets	Normal Form
1	DEPARTMENT	DeptID	DeptName, HeadEmpID	HeadEmpID → EMPLOYEE(EmpID)	**3NF**
2	EMPLOYEE	EmpID	Email	DeptID → DEPARTMENT, SupervisorID → EMPLOYEE	**3NF**
3	EMPLOYEE_PHONE	(EmpID, PhoneNumber)	*None*	EmpID → EMPLOYEE	**3NF**
4	EMPLOYEE_CERTIFICATION	(EmpID, Certification)	*None*	EmpID → EMPLOYEE	**3NF**
5	DEPENDENT	(EmpID, DependentName)	*None*	EmpID → EMPLOYEE	**3NF**
6	CLIENT	ClientID	TaxCode	*None*	**3NF**
7	CLIENT_PHONE	(ClientID, ContactPhone)	*None*	ClientID → CLIENT	**3NF**
8	PROJECT	ProjectID	ProjectName	ClientID → CLIENT, DeptID → DEPARTMENT, DirectorEmpID → EMPLOYEE	**3NF**
9	PROJECT_SITE	SiteID	(ProjectID, SiteName), (Latitude, Longitude)	ProjectID → PROJECT	**3NF**
10	PROJECT_PHASE	(ProjectID, PhaseNumber)	*None*	ProjectID → PROJECT	**3NF**
11	TASK	TaskID	(ProjectID, PhaseNumber, TaskName)	(ProjectID, PhaseNumber) → PROJECT_PHASE	**3NF**
12	PRECEDES	(PredecessorTaskID, SuccessorTaskID)	*None*	PredecessorTaskID → TASK, SuccessorTaskID → TASK	**3NF**
13	WORKS_ON_TASK	(EmpID, TaskID)	*None*	EmpID → EMPLOYEE, TaskID → TASK	**3NF**
14	EQUIPMENT	EquipmentID	SerialNumber	*None*	**3NF**
15	MAINTENANCE_RECORD	(EquipmentID, RecordNumber)	*None*	EquipmentID → EQUIPMENT	**3NF**
16	EQUIPMENT_DEPLOYMENT	(EquipmentID, DeploymentStartDate)	*None*	EquipmentID → EQUIPMENT, SiteID → PROJECT_SITE, OperatorEmpID → EMPLOYEE	**3NF**
17	SUPPLIER	SupplierID	TaxCode	*None*	**3NF**
18	SUPPLIER_PHONE	(SupplierID, ContactPhone)	*None*	SupplierID → SUPPLIER	**3NF**
19	SUPPLIER_CATEGORY	(SupplierID, SupplyCategory)	*None*	SupplierID → SUPPLIER	**3NF**
20	MATERIAL	MaterialID	MaterialName	*None*	**3NF**
21	PURCHASE_ORDER	PO_ID	*None*	ProjectID → PROJECT, SupplierID → SUPPLIER	**3NF**
22	PO_ITEM	(PO_ID, LineNumber)	(PO_ID, MaterialID)	PO_ID → PURCHASE_ORDER, MaterialID → MATERIAL	**3NF**
23	SITE_INVENTORY	(SiteID, MaterialID)	*None*	SiteID → PROJECT_SITE, MaterialID → MATERIAL	**3NF**
24	DELIVERY_DISPATCH	DeliveryCode	*None*	MaterialID → MATERIAL, SiteID → PROJECT_SITE,	**3NF**

No.	Relational Schema Name	Primary Key (PK)	Alternate Candidate Keys (AK)	Foreign Keys (FK) & Referencing Targets	Normal Form
				ReceivedByEmpID → EMPLOYEE, PO_ID → PURCHASE_ORDER	
25	MATERIAL_USAGE	(TaskID, MaterialID, SiteID, UsageDate)	*None*	TaskID → TASK, (SiteID, MaterialID) → SITE_INVENTORY	**3NF**
26	SUBCONTRACTOR	SubcontractorID	TaxCode, LicenseNumber	*None*	**3NF**
27	SUBCONTRACT	ContractAgreementNo	*None*	ProjectID → PROJECT, SubcontractorID → SUBCONTRACTOR	**3NF**
28	SAFETY_INSPECTION	InspectionID	*None*	SiteID → PROJECT_SITE, InspectorEmpID → EMPLOYEE	**3NF**
29	INCIDENT_REPORT	IncidentID	*None*	SiteID → PROJECT_SITE, InvolvedEmpID → EMPLOYEE, DamagedEquipmentID → EQUIPMENT	**3NF**

3. PHYSICAL DATABASE DESIGN & IMPLEMENTATION DETAILS

3.1 DBMS Selection & Configuration

We implemented the physical database on **MySQL Community Server 8.0.46** with **MySQL Workbench 8.0**:

- **InnoDB Engine:** Supports ACID transactions and foreign-key referential integrity.

- **CHECK Constraints:** Validates domain rules such as positive salaries, working hours, and date ranges.

- **Recursive CTEs (WITH RECURSIVE):** Handles employee reporting hierarchies and DAG cycle detection on PRECEDES.

- **Triggers:** Enforces cross-table business rules and automated emergency alerts.

- **UTF-8 Support (utf8mb4):** Stores Vietnamese and multilingual text.

3.2 Database Scripts Organization & Proofs.

3.2.1 01_schema_ddl.sql (DDL — 29 Core Tables & Indexes):

- Creates A34Build_DB and all 29 normalized 3NF tables.
- Resolves the circular foreign key between DEPARTMENT.HeadEmpID and EMPLOYEE.DeptID using a deferred ALTER TABLE DEPARTMENT ADD CONSTRAINT FK_DEPARTMENT_HeadEmpID ... statement.
- Adds performance indexes on frequently joined/filtered foreign keys (IDX_EMPLOYEE_DeptID, IDX_PROJECT_ClientID, IDX_INCIDENT_REPORT_Site, etc.).
- Example code:

```
CREATE TABLE DEPARTMENT (
    DeptID VARCHAR(10) NOT NULL,
    DeptName VARCHAR(100) NOT NULL,
    OfficeLocation VARCHAR(255) NOT NULL,
    Budget DECIMAL(15,2) NOT NULL DEFAULT 0.00,
    HeadEmpID VARCHAR(10) NULL,
    CONSTRAINT PK_DEPARTMENT PRIMARY KEY (DeptID),
    CONSTRAINT UQ_DEPARTMENT_DeptName UNIQUE (DeptName),
    CONSTRAINT UQ_DEPARTMENT_HeadEmpID UNIQUE (HeadEmpID),
    CONSTRAINT CK_DEPARTMENT_Budget CHECK (Budget >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
…
```

```
-- Resolve circular FK from DEPARTMENT to EMPLOYEE

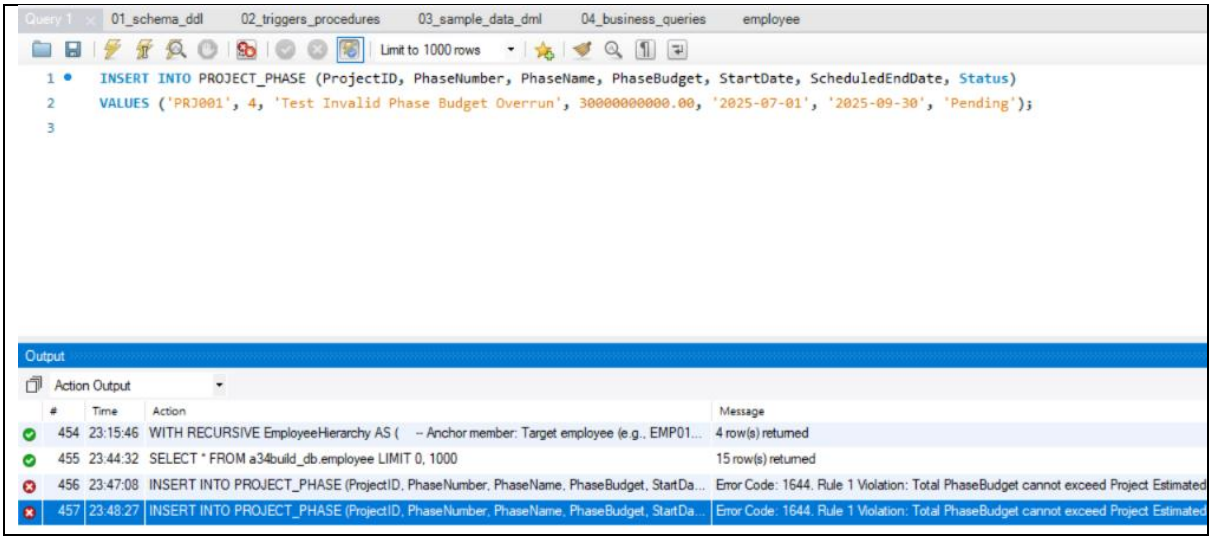
ALTER TABLE DEPARTMENT
    ADD CONSTRAINT FK_DEPARTMENT_HeadEmpID FOREIGN KEY (HeadEmpID)
    REFERENCES EMPLOYEE (EmpID)
    ON UPDATE CASCADE ON DELETE SET NULL;
```

The complete SQL scripts (01_schema_ddl.sql – 04_business_queries.sql) are attached in deliverables/sql/

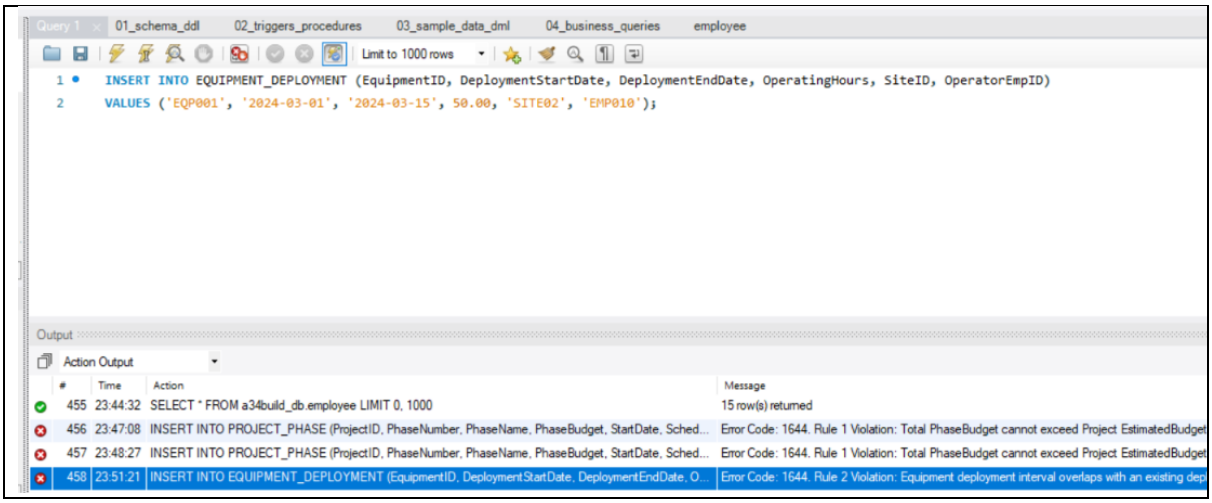
3.2.2 02_triggers_procedures.sql (8 Business Rule Triggers):

- Creates EMERGENCY_DISPATCH_LOG and registers triggers enforcing all **8 complex business rules** identified in Phase 2:

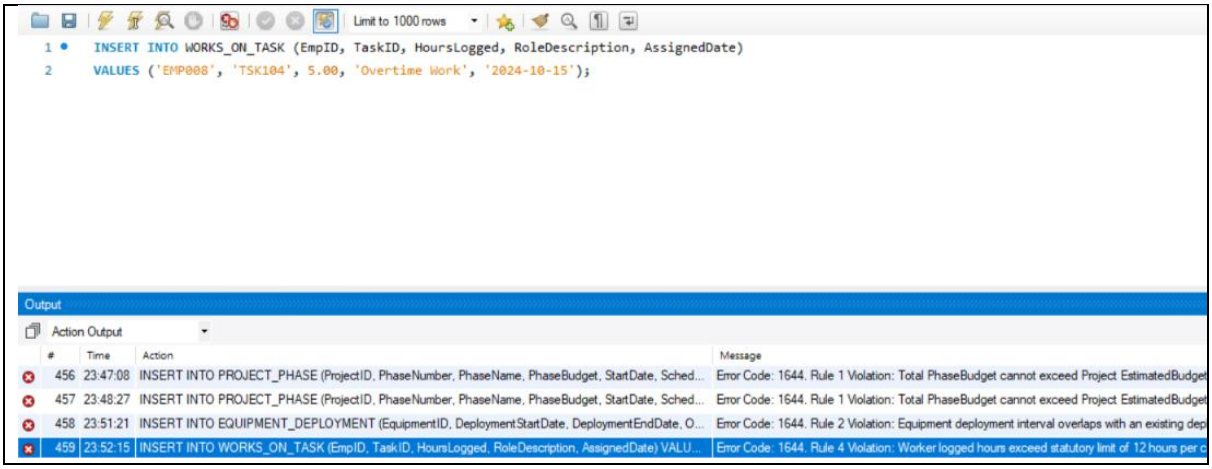
- **Rule 1:** Total PhaseBudget cannot exceed PROJECT.EstimatedBudget.



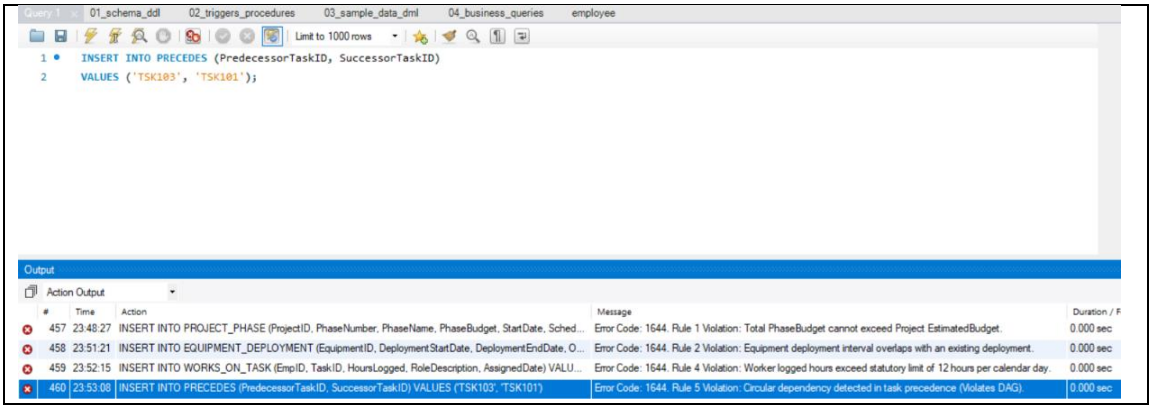
- **Rule 2:** No overlapping EQUIPMENT_DEPLOYMENT date intervals for the same machine.



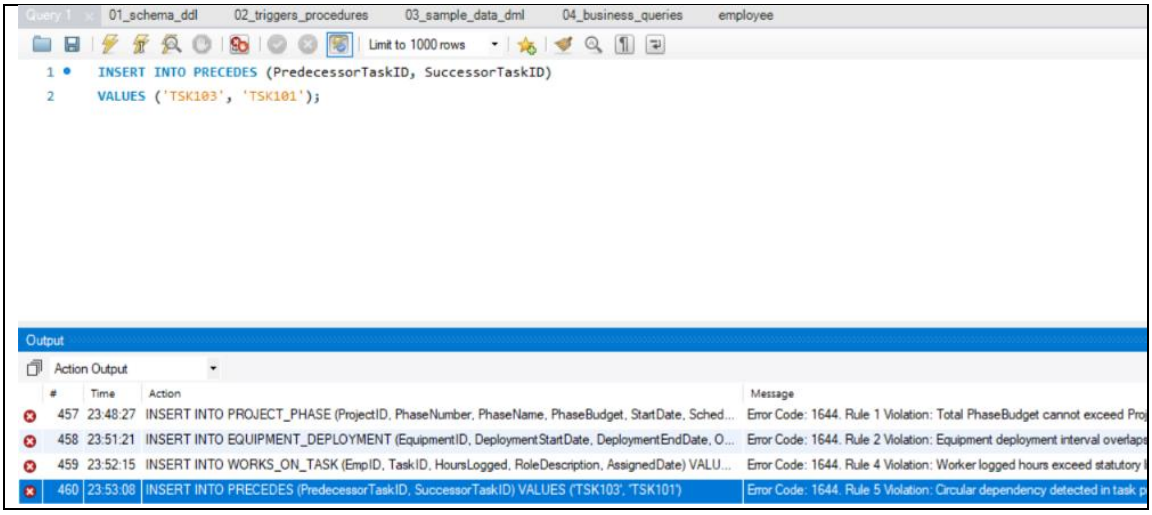
- **Rule 3:** Equipment operator must hold a valid certification matching the equipment category.



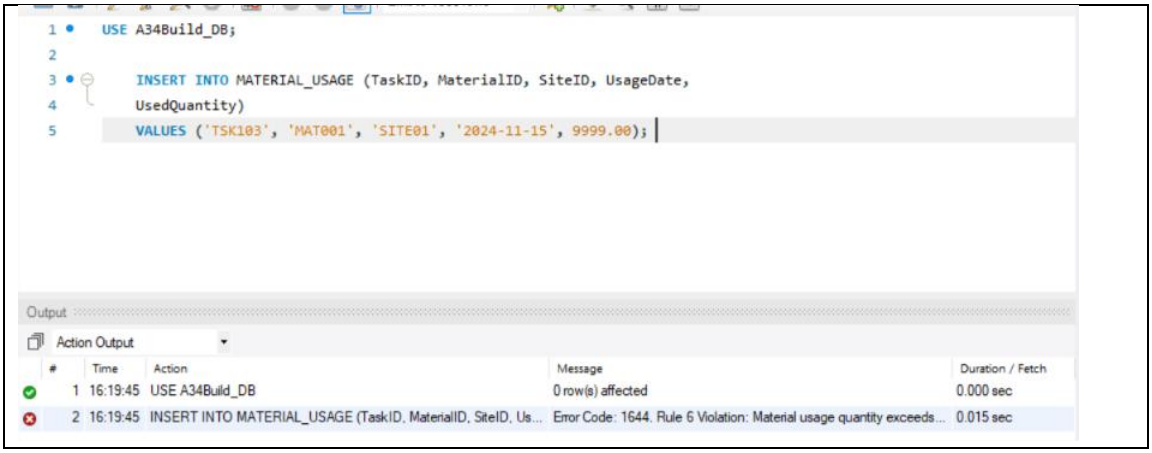
- **Rule 4:** Worker logged hours cannot exceed 12.00 hours per calendar day.



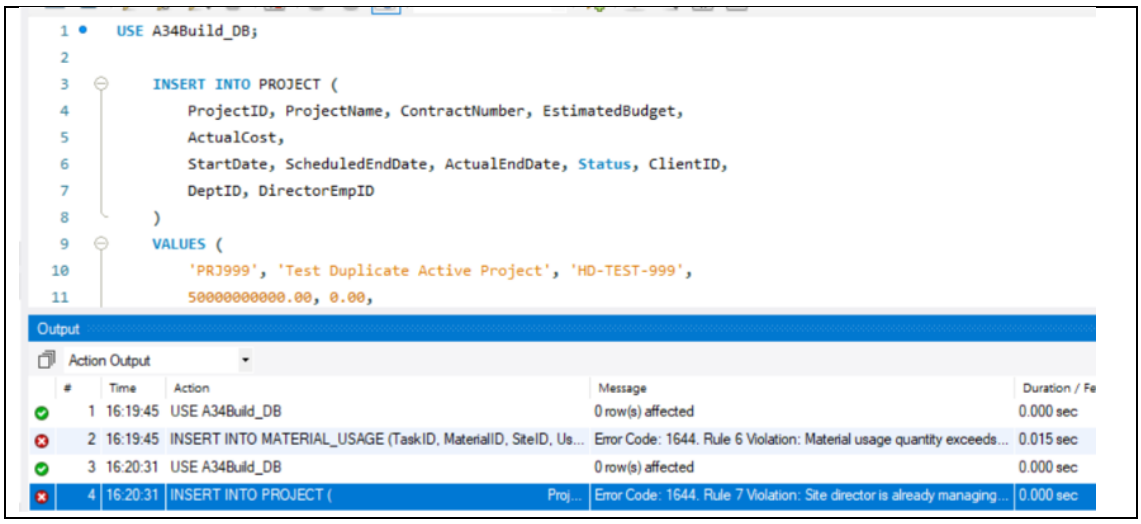
- **Rule 5:** PRECEDES task dependencies must form a DAG (no self-loops or cycles, checked via WITH RECURSIVE).



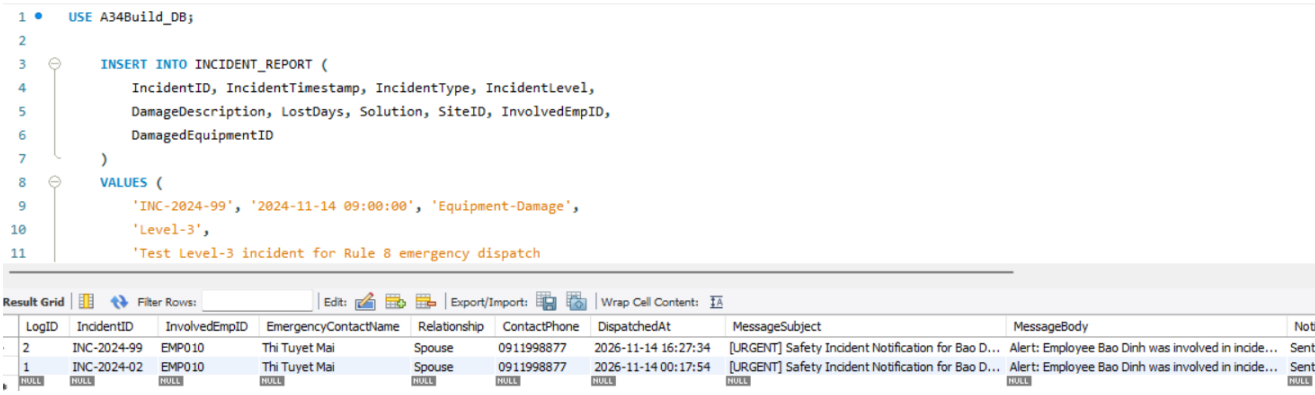
- **Rule 6:** Material usage cannot exceed SITE_INVENTORY.CurrentStockQuantity (automatically deducts stock).



- **Rule 7:** An employee can direct at most one project with Status = 'Active'.



- **Rule 8:** Level-2 to Level-4 incidents automatically look up the injured employee's DEPENDENT and log an emergency dispatch notification in EMERGENCY_DISPATCH_LOG.



- **Sample code:**

```
-- BUSINESS RULE 5: Acyclic Dependency Constraint (DAG on PRECEDES)
-- Prevents circular dependency loops (e.g., T1 -> T2 -> T3 -> T1).
=====================================================================
DROP TRIGGER IF EXISTS TRG_CHECK_PRECEDES_DAG //
CREATE TRIGGER TRG_CHECK_PRECEDES_DAG
BEFORE INSERT ON PRECEDES
FOR EACH ROW
BEGIN
    DECLARE v_cycle_detected INT DEFAULT 0;

    -- Self-loop check is already partially enforced, but verify
    IF NEW.PredecessorTaskID = NEW.SuccessorTaskID THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Rule 5 Violation: Self-loop precedence is strictly prohibited.';
    END IF;

    -- Check if a path already exists from SuccessorTaskID to PredecessorTaskID
    -- Using CTE recursion in MySQL 8.0+
    WITH RECURSIVE PrecedenceChain AS (
        SELECT SuccessorTaskID AS CurrentTask
        FROM PRECEDES
        WHERE PredecessorTaskID = NEW.SuccessorTaskID
```

```
        UNION ALL

        SELECT p.SuccessorTaskID
        FROM PRECEDES p
        INNER JOIN PrecedenceChain pc ON p.PredecessorTaskID = pc.CurrentTask
    )
    SELECT COUNT(*) INTO v_cycle_detected
    FROM PrecedenceChain
    WHERE CurrentTask = NEW.PredecessorTaskID;

    IF v_cycle_detected > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Rule 5 Violation: Circular dependency detected in task precedence
(Violates DAG).';
    END IF;
END //
```

```
-- ============================================================================
-- BUSINESS RULE 8: Automated Emergency Contact Notification on Incident
-- Finds dependent where IsEmergencyContact = TRUE and dispatches an alert.
-- ============================================================================
DROP TRIGGER IF EXISTS TRG_DISPATCH_EMERGENCY_ALERT //
CREATE TRIGGER TRG_DISPATCH_EMERGENCY_ALERT
AFTER INSERT ON INCIDENT_REPORT
FOR EACH ROW
BEGIN
    DECLARE v_contact_name VARCHAR(100);
    DECLARE v_relationship VARCHAR(50);
    DECLARE v_contact_phone VARCHAR(20);
    DECLARE v_emp_name VARCHAR(150);

    IF NEW.InvolvedEmpID IS NOT NULL AND NEW.IncidentLevel IN ('Level-2', 'Level-3', 'Level-
4') THEN
        -- Get Employee Name
        SELECT CONCAT(FirstName, ' ', LastName) INTO v_emp_name
        FROM EMPLOYEE
        WHERE EmpID = NEW.InvolvedEmpID;

        -- Find Emergency Dependent
        SELECT d.DependentName, d.Relationship, COALESCE(ep.PhoneNumber, 'No Phone on
Record')
        INTO v_contact_name, v_relationship, v_contact_phone
        FROM DEPENDENT d
        LEFT JOIN EMPLOYEE_PHONE ep ON ep.EmpID = d.EmpID
        WHERE d.EmpID = NEW.InvolvedEmpID
          AND d.IsEmergencyContact = TRUE
        LIMIT 1;

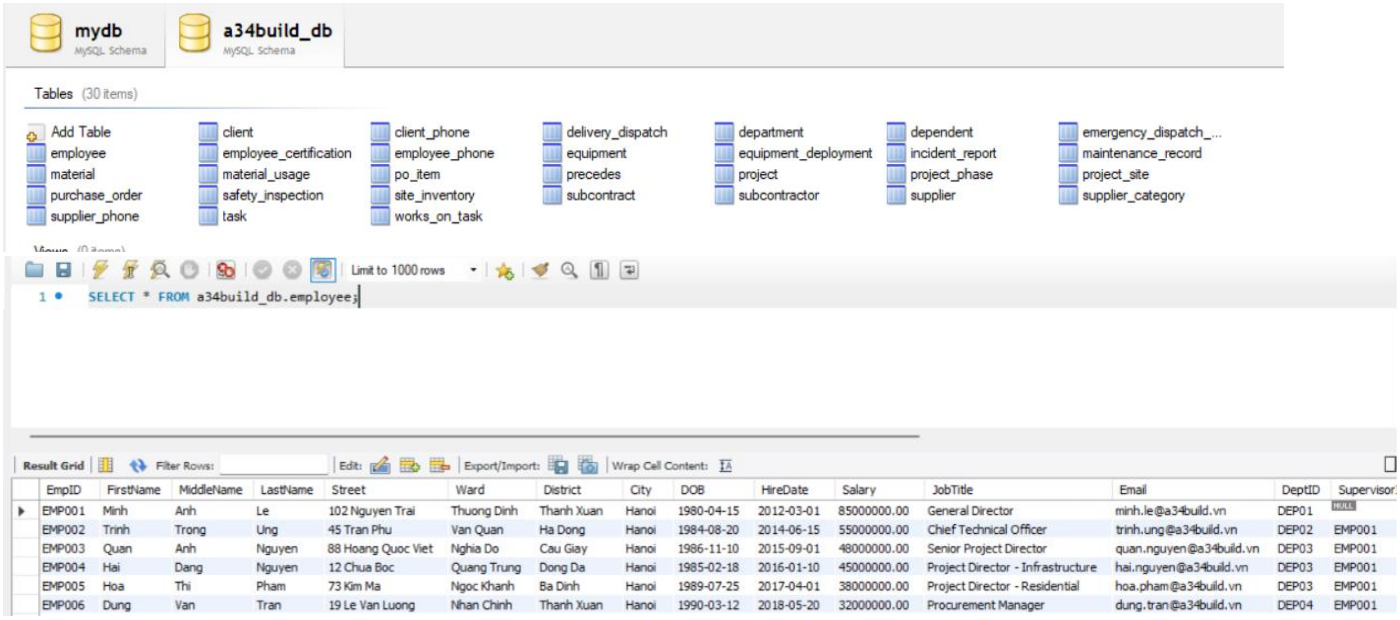
        -- If found, record into emergency dispatch log
        IF v_contact_name IS NOT NULL THEN
            INSERT INTO EMERGENCY_DISPATCH_LOG (
                IncidentID,
                InvolvedEmpID,
                EmergencyContactName,
                Relationship,
                ContactPhone,
```

```
                DispatchedAt,
                MessageSubject,
                MessageBody,
                NotificationStatus
            )
            VALUES (
                NEW.IncidentID,
                NEW.InvolvedEmpID,
                v_contact_name,
                v_relationship,
                v_contact_phone,
                NOW(),
                CONCAT('[URGENT] Safety Incident Notification for ', v_emp_name),
                CONCAT('Alert: Employee ', v_emp_name, ' was involved in incident ',
NEW.IncidentID,
                       ' (Level: ', NEW.IncidentLevel, ', Type: ', NEW.IncidentType, ') at
Site ', NEW.SiteID,
                       '. Description: ', NEW.DamageDescription),
                'Sent'
            );
        END IF;
    END IF;
END //
```

The complete SQL scripts (01_schema_ddl.sql – 04_business_queries.sql) are attached in deliverables/sql/

3.2.3 03_sample_data_dml.sql (Realistic Sample Data):

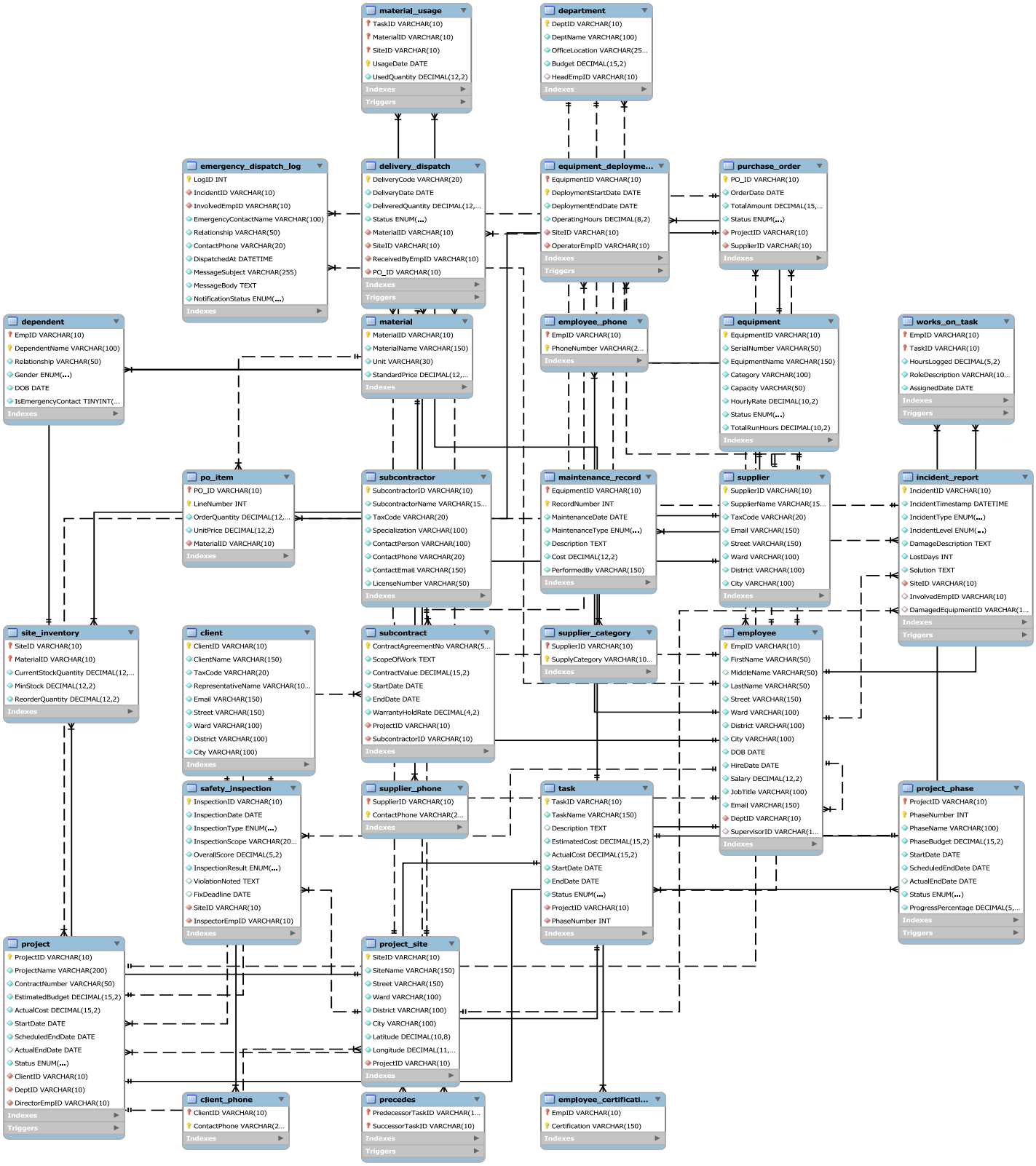
- Populates all 29 tables with realistic Vietnamese construction project data (projects in Hanoi, Da Nang, and Ho Chi Minh City; 12+ employees, equipment fleet, POs, site inventory, subcontractors, safety inspections, and incident reports).



EmpID	FirstName	MiddleName	LastName	Street	Ward	District	City	DOB	HireDate	Salary	JobTitle	Email	DeptID	Supervisor
EMP001	Minh	Anh	Le	102 Nguyen Trai	Thuong Dinh	Thanh Xuan	Hanoi	1980-04-15	2012-03-01	85000000.00	General Director	minh.le@a34build.vn	DEP01	NULL
EMP002	Trinh	Trong	Ung	45 Tran Phu	Van Quan	Ha Dong	Hanoi	1984-08-20	2014-06-15	55000000.00	Chief Technical Officer	trinh.ung@a34build.vn	DEP02	EMP001
EMP003	Quan	Anh	Nguyen	88 Hoang Quoc Viet	Nghia Do	Cau Giay	Hanoi	1986-11-10	2015-09-01	48000000.00	Senior Project Director	quan.nguyen@a34build.vn	DEP03	EMP001
EMP004	Hai	Dang	Nguyen	12 Chua Boc	Quang Trung	Dong Da	Hanoi	1985-02-18	2016-01-10	45000000.00	Project Director - Infrastructure	hai.nguyen@a34build.vn	DEP03	EMP001
EMP005	Hoa	Thi	Pham	73 Kim Ma	Ngoc Khanh	Ba Dinh	Hanoi	1989-07-25	2017-04-01	38000000.00	Project Director - Residential	hoa.pham@a34build.vn	DEP03	EMP001
EMP006	Dung	Van	Tran	19 Le Van Luong	Nhan Chinh	Thanh Xuan	Hanoi	1990-03-12	2018-05-20	32000000.00	Procurement Manager	dung.tran@a34build.vn	DEP04	EMP001

The complete SQL scripts (01_schema_ddl.sql – 04_business_queries.sql) are attached in deliverables/sql/

- EER diagram of database:


material_usage
TaskID VARCHAR(10)
MaterialID VARCHAR(10)
SiteID VARCHAR(10)
UsageDate DATE
UsedQuantity DECIMAL(12,2)
Indexes
Triggers
department
DeptID VARCHAR(10)
DeptName VARCHAR(100)
OfficeLocation VARCHAR(25...
Budget DECIMAL(15,2)
HeadEmpID VARCHAR(10)
Indexes
emergency_dispatch_log
LogID INT
IncidentID VARCHAR(10)
InvolvedEmpID VARCHAR(10)
EmergencyContactName VARCHAR(100)
Relationship VARCHAR(50)
ContactPhone VARCHAR(20)
DispatchedAt DATETIME
MessageSubject VARCHAR(255)
MessageBody TEXT
NotificationStatus ENUM(...)
Indexes
delivery_dispatch
DeliveryCode VARCHAR(20)
DeliveryDate DATE
DeliveredQuantity DECIMAL(12,...
Status ENUM(...)
MaterialID VARCHAR(10)
SiteID VARCHAR(10)
ReceivedByEmpID VARCHAR(10)
PO_ID VARCHAR(10)
Indexes
Triggers
equipment_deployme...
EquipmentID VARCHAR(10)
DeploymentStartDate DATE
DeploymentEndDate DATE
OperatingHours DECIMAL(8,2)
SiteID VARCHAR(10)
OperatorEmpID VARCHAR(10)
Indexes
Triggers
purchase_order
PO_ID VARCHAR(10)
OrderDate DATE
TotalAmount DECIMAL(15,...
Status ENUM(...)
ProjectID VARCHAR(10)
SupplierID VARCHAR(10)
Indexes
dependent
EmpID VARCHAR(10)
DependentName VARCHAR(100)
Relationship VARCHAR(50)
Gender ENUM(...)
DOB DATE
IsEmergencyContact TINYINT(...
Indexes
material
MaterialID VARCHAR(10)
MaterialName VARCHAR(150)
Unit VARCHAR(30)
StandardPrice DECIMAL(12,...
Indexes
employee_phone
EmpID VARCHAR(10)
PhoneNumber VARCHAR(2...
Indexes
equipment
EquipmentID VARCHAR(10)
SerialNumber VARCHAR(50)
EquipmentName VARCHAR(150)
Category VARCHAR(100)
Capacity VARCHAR(50)
HourlyRate DECIMAL(10,2)
Status ENUM(...)
TotalRunHours DECIMAL(10,2)
Indexes
works_on_task
EmpID VARCHAR(10)
TaskID VARCHAR(10)
HoursLogged DECIMAL(5,2)
RoleDescription VARCHAR(10...
AssignedDate DATE
Indexes
Triggers
po_item
PO_ID VARCHAR(10)
LineNumber INT
OrderQuantity DECIMAL(12,...
UnitPrice DECIMAL(12,2)
MaterialID VARCHAR(10)
Indexes
subcontractor
SubcontractorID VARCHAR(10)
SubcontractorName VARCHAR(15...
TaxCode VARCHAR(20)
Specialization VARCHAR(100)
ContactPerson VARCHAR(100)
ContactPhone VARCHAR(20)
ContactEmail VARCHAR(150)
LicenseNumber VARCHAR(50)
Indexes
maintenance_record
EquipmentID VARCHAR(10)
RecordNumber INT
MaintenanceDate DATE
MaintenanceType ENUM(...)
Description TEXT
Cost DECIMAL(12,2)
PerformedBy VARCHAR(150)
Indexes
supplier
SupplierID VARCHAR(10)
SupplierName VARCHAR(15...
TaxCode VARCHAR(20)
Email VARCHAR(150)
Street VARCHAR(150)
Ward VARCHAR(100)
District VARCHAR(100)
City VARCHAR(100)
Indexes
incident_report
IncidentID VARCHAR(10)
IncidentTimestamp DATETIME
IncidentType ENUM(...)
IncidentLevel ENUM(...)
DamageDescription TEXT
LostDays INT
Solution TEXT
SiteID VARCHAR(10)
InvolvedEmpID VARCHAR(10)
DamagedEquipmentID VARCHAR(1...
Indexes
Triggers
site_inventory
SiteID VARCHAR(10)
MaterialID VARCHAR(10)
CurrentStockQuantity DECIMAL(12,...
MinStock DECIMAL(12,2)
ReorderQuantity DECIMAL(12,2)
Indexes
client
ClientID VARCHAR(10)
ClientName VARCHAR(150)
TaxCode VARCHAR(20)
RepresentativeName VARCHAR(10...
Email VARCHAR(150)
Street VARCHAR(150)
Ward VARCHAR(100)
District VARCHAR(100)
City VARCHAR(100)
Indexes
subcontract
ContractAgreementNo VARCHAR(5...
ScopeOfWork TEXT
ContractValue DECIMAL(15,2)
StartDate DATE
EndDate DATE
WarrantyHoldRate DECIMAL(4,2)
ProjectID VARCHAR(10)
SubcontractorID VARCHAR(10)
Indexes
supplier_category
SupplierID VARCHAR(10)
SupplyCategory VARCHAR(10...
Indexes
employee
EmpID VARCHAR(10)
FirstName VARCHAR(50)
MiddleName VARCHAR(50)
LastName VARCHAR(50)
Street VARCHAR(150)
Ward VARCHAR(100)
District VARCHAR(100)
City VARCHAR(100)
DOB DATE
HireDate DATE
Salary DECIMAL(12,2)
JobTitle VARCHAR(100)
Email VARCHAR(150)
DeptID VARCHAR(10)
SupervisorID VARCHAR(1...
Indexes
safety_inspection
InspectionID VARCHAR(10)
InspectionDate DATE
InspectionType ENUM(...)
InspectionScope VARCHAR(20...
OverallScore DECIMAL(5,2)
InspectionResult ENUM(...)
ViolationNoted TEXT
FixDeadline DATE
SiteID VARCHAR(10)
InspectorEmpID VARCHAR(10)
Indexes
supplier_phone
SupplierID VARCHAR(10)
ContactPhone VARCHAR(2...
Indexes
task
TaskID VARCHAR(10)
TaskName VARCHAR(150)
Description TEXT
EstimatedCost DECIMAL(15,2)
ActualCost DECIMAL(15,2)
StartDate DATE
EndDate DATE
Status ENUM(...)
ProjectID VARCHAR(10)
PhaseNumber INT
Indexes
project_phase
ProjectID VARCHAR(10)
PhaseNumber INT
PhaseName VARCHAR(100)
PhaseBudget DECIMAL(15,2)
StartDate DATE
ScheduledEndDate DATE
ActualEndDate DATE
Status ENUM(...)
ProgressPercentage DECIMAL(5,...
Indexes
Triggers
project
ProjectID VARCHAR(10)
ProjectName VARCHAR(200)
ContractNumber VARCHAR(50)
EstimatedBudget DECIMAL(15,2)
ActualCost DECIMAL(15,2)
StartDate DATE
ScheduledEndDate DATE
ActualEndDate DATE
Status ENUM(...)
ClientID VARCHAR(10)
DeptID VARCHAR(10)
DirectorEmpID VARCHAR(10)
Indexes
Triggers
project_site
SiteID VARCHAR(10)
SiteName VARCHAR(150)
Street VARCHAR(150)
Ward VARCHAR(100)
District VARCHAR(100)
City VARCHAR(100)
Latitude DECIMAL(10,8)
Longitude DECIMAL(11,...
ProjectID VARCHAR(10)
Indexes
client_phone
ClientID VARCHAR(10)
ContactPhone VARCHAR(2...
Indexes
precedes
PredecessorTaskID VARCHAR(1...
SuccessorTaskID VARCHAR(10)
Indexes
Triggers
employee_certificati...
EmpID VARCHAR(10)
Certification VARCHAR(150)
Indexes

3.2.4 04_business_queries.sql (Operational & Analytical Queries):

Implements 9 queries/reports covering operational checks, recursive hierarchy, cost/budget variance, Earned Value Management (EVM), safety & incident audit, fleet utilization, and trigger verification. And

- *Query 1.1: Find machinery by category that is currently 'Available' or not deployed in the next 14 days.*

```
SELECT
    e.EquipmentID,
    e.EquipmentName,
    e.Category,
    e.Capacity,
    e.HourlyRate,
    e.Status AS CurrentStatus,
    COALESCE(MAX(ed.DeploymentEndDate), 'Never Deployed') AS LastDeploymentEndDate
FROM EQUIPMENT e
LEFT JOIN EQUIPMENT_DEPLOYMENT ed ON e.EquipmentID = ed.EquipmentID
WHERE e.Status NOT IN ('Decommissioned', 'Under-Maintenance')
  AND e.EquipmentID NOT IN (
      SELECT EquipmentID
      FROM EQUIPMENT_DEPLOYMENT
      WHERE DeploymentEndDate >= CURDATE()
        AND DeploymentStartDate <= DATE_ADD(CURDATE(), INTERVAL 14 DAY)
  )
GROUP BY e.EquipmentID, e.EquipmentName, e.Category, e.Capacity, e.HourlyRate, e.Status
ORDER BY e.HourlyRate DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

EquipmentID	EquipmentName	Category	Capacity	HourlyRate	CurrentStatus	LastDeploymentEndDate
EQP002	Liebherr Crawler Crane LR 1300	Heavy Crane	300 Tons	3500000.00	Available	Never Deployed
EQP001	Potain Top-Slewing Tower Crane MD 345	Tower Crane	12 Tons	1850000.00	Deployed	2024-12-31
EQP004	Putzmeister Truck-Mounted Concrete Pump 47Z	Concrete Pump	160 m3/h	1400000.00	Available	Never Deployed
EQP003	Caterpillar Hydraulic Excavator 336D	Excavator	2.5 m3	950000.00	Deployed	2024-11-15

- *Query 1.2: Check materials whose current stock on site is below or near the minimum safety stock.*

```
SELECT
    ps.SiteID,
    ps.SiteName,
    p.ProjectName,
    m.MaterialID,
    m.MaterialName,
    m.Unit,
    si.CurrentStockQuantity,
    si.MinStock,
    si.ReorderQuantity,
    (si.MinStock - si.CurrentStockQuantity) AS DeficitQuantity,
    ROUND((si.CurrentStockQuantity / NULLIF(si.MinStock, 0)) * 100, 2) AS
StockSufficiencyRatioPct,
    CASE
        WHEN si.CurrentStockQuantity <= (si.MinStock * 0.5) THEN 'CRITICAL: Stockout Imminent'
        WHEN si.CurrentStockQuantity <= si.MinStock THEN 'WARNING: Below Min Safety Stock'
        ELSE 'NORMAL: Adequate'
    END AS ReplenishmentStatus
FROM SITE_INVENTORY si
```

```
JOIN PROJECT_SITE ps ON si.SiteID = ps.SiteID
JOIN PROJECT p ON ps.ProjectID = p.ProjectID
JOIN MATERIAL m ON si.MaterialID = m.MaterialID
WHERE si.CurrentStockQuantity <= (si.MinStock * 1.5)
ORDER BY StockSufficiencyRatioPct ASC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

SiteID	SiteName	ProjectName	MaterialID	MaterialName	Unit	CurrentStockQuantity	MinStock	ReorderQuantity	DeficitQuantity	StockSufficiencyRatioPct	ReplenishmentStatus
SITE01	Landmark North Tower Base Site	Landmark A34 Twin Towers Complex	MAT003	Portland Blended Cement PCB40 Bulk	Ton	18.00	40.00	100.00	22.00	45.00	CRITICAL: Stockout Imminent
SITE03	Coastal Bridge Pier Pier-14 to Pier-22	Coastal Highway Bridge Overpass	MAT002	High-Tensile Prestressing Strand 15.2mm	Ton	60.00	80.00	100.00	20.00	75.00	WARNING: Below Min Safety Stock
SITE03	Coastal Bridge Pier Pier-14 to Pier-22	Coastal Highway Bridge Overpass	MAT005	High-Strength River Sand (Washed)	m3	110.00	80.00	300.00	-30.00	137.50	NORMAL: Adequate

- *Query 1.3: Look up the complete management reporting chain of an employee up to the General Director.*

```
WITH RECURSIVE EmployeeHierarchy AS (
    -- Anchor member: Target employee (e.g., EMP010 - Crane Operator)
    SELECT
        EmpID,
        FirstName,
        LastName,
        JobTitle,
        SupervisorID,
        1 AS HierarchyLevel,
        CAST(CONCAT(FirstName, ' ', LastName) AS CHAR(500)) AS ManagementPath
    FROM EMPLOYEE
    WHERE EmpID = 'EMP010'

    UNION ALL

    -- Recursive member: Traverse up to the supervisor
    SELECT
        m.EmpID,
        m.FirstName,
        m.LastName,
        m.JobTitle,
        m.SupervisorID,
        eh.HierarchyLevel + 1,
        CONCAT(eh.ManagementPath, ' -> ', m.FirstName, ' ', m.LastName)
    FROM EMPLOYEE m
    INNER JOIN EmployeeHierarchy eh ON eh.SupervisorID = m.EmpID
)
SELECT
    HierarchyLevel,
    EmpID,
    CONCAT(FirstName, ' ', LastName) AS EmployeeName,
    JobTitle,
    SupervisorID,
    ManagementPath
FROM EmployeeHierarchy
ORDER BY HierarchyLevel ASC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

HierarchyLevel	EmpID	EmployeeName	JobTitle	SupervisorID	ManagementPath
1	EMP010	Bao Dinh	Master Heavy Crane Operator	EMP008	Bao Dinh
2	EMP008	Nam Vu	Site Lead Engineer	EMP003	Bao Dinh -> Nam Vu
3	EMP003	Quan Nguyen	Senior Project Director	EMP001	Bao Dinh -> Nam Vu -> Quan Nguyen
4	EMP001	Minh Le	General Director	NULL	Bao Dinh -> Nam Vu -> Quan Nguyen -> Minh Le

- *Query 1.4: Calculate financial commitments and warranty retention hold amounts per subcontractor.*

```
SELECT
    s.SubcontractorID,
    s.SubcontractorName,
    s.Specialization,
    sc.ContractAgreementNo,
    p.ProjectName,
    sc.ContractValue,
    sc.WarrantyHoldRate,
    ROUND((sc.ContractValue * sc.WarrantyHoldRate / 100.0), 2) AS WarrantyRetainedAmount,
    ROUND(sc.ContractValue - (sc.ContractValue * sc.WarrantyHoldRate / 100.0), 2) AS
NetPayableAmount,
    sc.StartDate,
    sc.EndDate,
    DATEDIFF(sc.EndDate, CURDATE()) AS DaysRemaining
FROM SUBCONTRACT sc
JOIN SUBCONTRACTOR s ON sc.SubcontractorID = s.SubcontractorID
JOIN PROJECT p ON sc.ProjectID = p.ProjectID
WHERE sc.EndDate >= CURDATE()
ORDER BY sc.ContractValue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

SubcontractorID	SubcontractorName	Specialization	ContractAgreementNo	ProjectName	ContractValue	WarrantyHoldRate	WarrantyRetainedAmount	NetPayableAmount	StartDate	EndDate	DaysRema
SUB03	Hoa Binh Prestressing Technology JSC	Post-Tensioning & Bridge Girders	HD-SUB-2024-08	Coastal Highway Bridge Overpass	18500000000.00	7.50	1387500000.00	17112500000.00	2024-06-01	2027-03-14	120
SUB03	Hoa Binh Prestressing Technology JSC	Post-Tensioning & Bridge Girders	HD-SUB-2024-04	Landmark A34 Twin Towers Complex	14000000000.00	5.00	700000000.00	13300000000.00	2024-05-01	2026-12-29	45

- *Report 2.1: Aggregates budget breakdown and compares with actual costs (Labor, Machinery, Materials, Subcontracts).*

```
WITH LaborCost AS (
    SELECT
        t.ProjectID,
        COALESCE(SUM(w.HoursLogged * (e.Salary / 160.0)), 0) AS TotalLaborCost
    FROM TASK t
    JOIN WORKS_ON_TASK w ON t.TaskID = w.TaskID
    JOIN EMPLOYEE e ON w.EmpID = e.EmpID
    GROUP BY t.ProjectID
),
EquipmentCost AS (
    SELECT
        ps.ProjectID,
        COALESCE(SUM(ed.OperatingHours * eq.HourlyRate), 0) AS TotalEquipmentCost
    FROM PROJECT_SITE ps
    JOIN EQUIPMENT_DEPLOYMENT ed ON ps.SiteID = ed.SiteID
    JOIN EQUIPMENT eq ON ed.EquipmentID = eq.EquipmentID
    GROUP BY ps.ProjectID
),
MaterialCost AS (
    SELECT
        t.ProjectID,
        COALESCE(SUM(mu.UsedQuantity * m.StandardPrice), 0) AS TotalMaterialCost
    FROM TASK t
    JOIN MATERIAL_USAGE mu ON t.TaskID = mu.TaskID
    JOIN MATERIAL m ON mu.MaterialID = m.MaterialID
    GROUP BY t.ProjectID
```

```
),
SubcontractCost AS (
    SELECT
        ProjectID,
        COALESCE(SUM(ContractValue), 0) AS TotalSubcontractCost
    FROM SUBCONTRACT
    GROUP BY ProjectID
)
SELECT
    p.ProjectID,
    p.ProjectName,
    p.Status,
    p.EstimatedBudget,
    p.ActualCost AS GeneralLedgerActualCost,
    ROUND(COALESCE(lc.TotalLaborCost, 0), 2) AS CalculatedLaborCost,
    ROUND(COALESCE(ec.TotalEquipmentCost, 0), 2) AS CalculatedEquipmentCost,
    ROUND(COALESCE(mc.TotalMaterialCost, 0), 2) AS CalculatedMaterialCost,
    ROUND(COALESCE(sc.TotalSubcontractCost, 0), 2) AS CommittedSubcontractCost,
    ROUND(
        COALESCE(lc.TotalLaborCost, 0) +
        COALESCE(ec.TotalEquipmentCost, 0) +
        COALESCE(mc.TotalMaterialCost, 0) +
        COALESCE(sc.TotalSubcontractCost, 0), 2
    ) AS TotalProjectCommittedCost,
    ROUND(
        p.EstimatedBudget - (
            COALESCE(lc.TotalLaborCost, 0) +
            COALESCE(ec.TotalEquipmentCost, 0) +
            COALESCE(mc.TotalMaterialCost, 0) +
            COALESCE(sc.TotalSubcontractCost, 0)
        ), 2
    ) AS RemainingBudgetVariance,
    ROUND(
        ((COALESCE(lc.TotalLaborCost, 0) + COALESCE(ec.TotalEquipmentCost, 0) +
          COALESCE(mc.TotalMaterialCost, 0) + COALESCE(sc.TotalSubcontractCost, 0)) /
NULLIF(p.EstimatedBudget, 0)) * 100, 2
    ) AS BudgetBurnRatePct
FROM PROJECT p
LEFT JOIN LaborCost lc ON p.ProjectID = lc.ProjectID
LEFT JOIN EquipmentCost ec ON p.ProjectID = ec.ProjectID
LEFT JOIN MaterialCost mc ON p.ProjectID = mc.ProjectID
LEFT JOIN SubcontractCost sc ON p.ProjectID = sc.ProjectID
ORDER BY p.EstimatedBudget DESC;
```

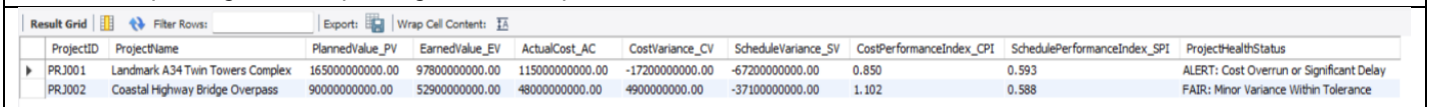
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ectID	ProjectName	Status	EstimatedBudget	GeneralLedgerActualCost	CalculatedLaborCost	CalculatedEquipmentCost	CalculatedMaterialCost	CommittedSubcontractCost	TotalProjectCommittedCost	RemainingBudgetVariance	BudgetBurnR
001	Landmark A34 Twin Towers Complex	Active	180000000000.00	115000000000.00	6390625.00	3145000000.00	883500000.00	40000000000.00	44034890625.00	135965109375.00	24.46
002	Coastal Highway Bridge Overpass	Active	95000000000.00	48000000000.00	1000000.00	807500000.00	288000000.00	18500000000.00	19596500000.00	75403500000.00	20.63
004	Masterise Riverside Luxury Villas	Completed	75000000000.00	73500000000.00	0.00	0.00	0.00	0.00	0.00	75000000000.00	0.00
003	PTIT Smart Digital Technology Center	Planning	45000000000.00	2000000000.00	0.00	0.00	0.00	0.00	0.00	45000000000.00	0.00

- *Report 2.2:Computes Planned Value (PV), Earned Value (EV), Actual Cost (AC), CPI, and SPI.*

```
SELECT
    p.ProjectID,
    p.ProjectName,
```

```
    SUM(pp.PhaseBudget) AS PlannedValue_PV,
    ROUND(SUM(pp.PhaseBudget * (pp.ProgressPercentage / 100.0)), 2) AS EarnedValue_EV,
    p.ActualCost AS ActualCost_AC,
    -- Cost Variance (CV = EV - AC)
    ROUND(SUM(pp.PhaseBudget * (pp.ProgressPercentage / 100.0)) - p.ActualCost, 2) AS
CostVariance_CV,
    -- Schedule Variance (SV = EV - PV)
    ROUND(SUM(pp.PhaseBudget * (pp.ProgressPercentage / 100.0)) - SUM(pp.PhaseBudget), 2) AS
ScheduleVariance_SV,
    -- Cost Performance Index (CPI = EV / AC)
    ROUND(SUM(pp.PhaseBudget * (pp.ProgressPercentage / 100.0)) / NULLIF(p.ActualCost, 0), 3)
AS CostPerformanceIndex_CPI,
    -- Schedule Performance Index (SPI = EV / PV)
    ROUND(SUM(pp.PhaseBudget * (pp.ProgressPercentage / 100.0)) / NULLIF(SUM(pp.PhaseBudget),
0), 3) AS SchedulePerformanceIndex_SPI,
    -- Project Health Status
    CASE
        WHEN (SUM(pp.PhaseBudget * (pp.ProgressPercentage / 100.0)) / NULLIF(p.ActualCost, 0))
>= 1.0
         AND (SUM(pp.PhaseBudget * (pp.ProgressPercentage / 100.0)) /
NULLIF(SUM(pp.PhaseBudget), 0)) >= 1.0
        THEN 'EXCELLENT: Ahead of Schedule & Under Budget'
        WHEN (SUM(pp.PhaseBudget * (pp.ProgressPercentage / 100.0)) / NULLIF(p.ActualCost, 0))
>= 0.90
        THEN 'FAIR: Minor Variance Within Tolerance'
        ELSE 'ALERT: Cost Overrun or Significant Delay'
    END AS ProjectHealthStatus
FROM PROJECT p
JOIN PROJECT_PHASE pp ON p.ProjectID = pp.ProjectID
WHERE p.Status = 'Active'
GROUP BY p.ProjectID, p.ProjectName, p.ActualCost;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProjectID	ProjectName	PlannedValue_PV	EarnedValue_EV	ActualCost_AC	CostVariance_CV	ScheduleVariance_SV	CostPerformanceIndex_CPI	SchedulePerformanceIndex_SPI	ProjectHealthStatus
PRJ001	Landmark A34 Twin Towers Complex	165000000000.00	97800000000.00	115000000000.00	-17200000000.00	-67200000000.00	0.850	0.593	ALERT: Cost Overrun or Significant Delay
PRJ002	Coastal Highway Bridge Overpass	90000000000.00	52900000000.00	48000000000.00	4900000000.00	-37100000000.00	1.102	0.588	FAIR: Minor Variance Within Tolerance

- *Report 2.3: Summarizes safety audit scores, open corrective actions, and lost-time injuries.*

```
SELECT
    ps.SiteID,
    ps.SiteName,
    p.ProjectName,
    COUNT(DISTINCT si.InspectionID) AS TotalInspectionsConducted,
    ROUND(AVG(si.OverallScore), 2) AS AverageSafetyAuditScore,
    COUNT(DISTINCT CASE WHEN si.InspectionResult = 'Pass' THEN si.InspectionID END) AS
PassedAudits,
    COUNT(DISTINCT CASE WHEN si.InspectionResult IN ('Conditional-Pass', 'Fail') THEN
si.InspectionID END) AS AuditsWithDeficiencies,
    COUNT(DISTINCT ir.IncidentID) AS TotalIncidentsRecorded,
    COALESCE(SUM(ir.LostDays), 0) AS TotalLostDaysFromInjuries,
    CASE
        WHEN COALESCE(SUM(ir.LostDays), 0) = 0 AND AVG(si.OverallScore) >= 90.0 THEN
'EXEMPLARY: Gold Standard Site'
        WHEN AVG(si.OverallScore) >= 75.0 THEN 'ACCEPTABLE: Standard Site Monitoring'
        ELSE 'HIGH RISK: Immediate Safety Intervention Required'
    END AS SiteRiskProfile
```

```
FROM PROJECT_SITE ps
JOIN PROJECT p ON ps.ProjectID = p.ProjectID
LEFT JOIN SAFETY_INSPECTION si ON ps.SiteID = si.SiteID
LEFT JOIN INCIDENT_REPORT ir ON ps.SiteID = ir.SiteID
GROUP BY ps.SiteID, ps.SiteName, p.ProjectName;
```

SiteName	ProjectName	TotalInspectionsConducted	AverageSafetyAuditScore	PassedAudits	AuditsWithDeficiencies	TotalIncidentsRecorded	TotalLostDaysFromInjuries	SiteRiskProfile
Landmark North Tower Base Site	Landmark A34 Twin Towers Complex	2	85.25	1	1	2	6	ACCEPTABLE: Standard Site Monitoring
Landmark South Commercial Podium	Landmark A34 Twin Towers Complex	0	NULL	0	0	0	0	HIGH RISK: Immediate Safety Intervention Req.
Coastal Bridge Pier Pier-14 to Pier-22	Coastal Highway Bridge Overpass	1	88.00	1	0	0	0	ACCEPTABLE: Standard Site Monitoring
PTIT Campus Innovation Site	PTIT Smart Digital Technology Center	0	NULL	0	0	0	0	HIGH RISK: Immediate Safety Intervention Req.

- *Report 2.4: Evaluates machine productivity (run hours vs maintenance expense).*

```
SELECT
    e.EquipmentID,
    e.EquipmentName,
    e.Category,
    e.HourlyRate,
    e.TotalRunHours,
    ROUND(e.TotalRunHours * e.HourlyRate, 2) AS LifetimeGrossRevenueGenerated,
    COUNT(mr.RecordNumber) AS TotalMaintenanceSessions,
    COALESCE(SUM(mr.Cost), 0.00) AS TotalMaintenanceCostSpent,
    ROUND(
        (COALESCE(SUM(mr.Cost), 0.00) / NULLIF(e.TotalRunHours * e.HourlyRate, 0)) * 100, 2
    ) AS MaintenanceCostToRevenueRatioPct
FROM EQUIPMENT e
LEFT JOIN MAINTENANCE_RECORD mr ON e.EquipmentID = mr.EquipmentID
GROUP BY e.EquipmentID, e.EquipmentName, e.Category, e.HourlyRate, e.TotalRunHours
ORDER BY LifetimeGrossRevenueGenerated DESC;
```

EquipmentID	EquipmentName	Category	HourlyRate	TotalRunHours	LifetimeGrossRevenueGenerated	TotalMaintenanceSessions	TotalMaintenanceCostSpent	MaintenanceCostToRevenueRatioPct
EQP002	Liebherr Crawler Crane LR 1300	Heavy Crane	3500000.00	1420.00	4970000000.00	0	0.00	0.00
EQP001	Potain Top-Slewing Tower Crane MD 345	Tower Crane	1850000.00	2140.50	3959925000.00	2	43000000.00	1.09
EQP003	Caterpillar Hydraulic Excavator 336D	Excavator	950000.00	3850.00	3657500000.00	1	12000000.00	0.33
EQP004	Putzmeister Truck-Mounted Concrete Pump 47Z	Concrete Pump	1400000.00	1650.00	2310000000.00	0	0.00	0.00
EQP005	Dynapac Double Drum Asphalt Roller CC6200	Asphalt Roller	750000.00	2890.00	2167500000.00	1	45000000.00	2.08

- *Verification Query 3.1: Verify Rule 8 Emergency Dispatch Trigger Execution*
- *Confirms that when INC-2024-02 was created, the trigger automatically identified the employee's dependent and logged an emergency alert.*

```
SELECT
    l.LogID,
    l.IncidentID,
    l.InvolvedEmpID,
    CONCAT(e.FirstName, ' ', e.LastName) AS InjuredEmployee,
    l.EmergencyContactName,
    l.Relationship,
    l.ContactPhone,
    l.DispatchedAt,
    l.MessageSubject,
    l.MessageBody,
    l.NotificationStatus
FROM EMERGENCY_DISPATCH_LOG l
JOIN EMPLOYEE e ON l.InvolvedEmpID = e.EmpID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

EquipmentID	EquipmentName	Category	HourlyRate	TotalRunHours	LifetimeGrossRevenueGenerated	TotalMaintenanceSessions	TotalMaintenanceCostSpent	MaintenanceCostToRevenueRatioPct
EQP002	Liebherr Crawler Crane LR 1300	Heavy Crane	3500000.00	1420.00	4970000000.00	0	0.00	0.00
EQP001	Potain Top-Slewing Tower Crane MD 345	Tower Crane	1850000.00	2140.50	3959925000.00	2	43000000.00	1.09
EQP003	Caterpillar Hydraulic Excavator 336D	Excavator	950000.00	3850.00	3657500000.00	1	12000000.00	0.33
EQP004	Putzmeister Truck-Mounted Concrete Pump 47Z	Concrete Pump	1400000.00	1650.00	2310000000.00	0	0.00	0.00
EQP005	Dynapac Double Drum Asphalt Roller CC6200	Asphalt Roller	750000.00	2890.00	2167500000.00	1	45000000.00	2.08

3.2.5 05_user_queries.sql (User Queries — Multi-Table Joins, Grouping & Filtering)

Implements 2 user-defined analytical queries demonstrating multi-table INNER JOIN, WHERE filtering, GROUP BY, HAVING, and ORDER BY, together with their corresponding Relational Algebra expressions and MySQL Workbench execution results.

Query 5.1: High-Salary Employee Workload by Department

Retrieves employees earning at least 15,000,000 VND who are assigned to project tasks, summarizing their assigned task count and total logged hours.

- **Problem Statement:** List employees with Salary >= 15,000,000 VND who are assigned to tasks, displaying their department, salary, total assigned tasks, and total logged hours. Keep only employees with total logged hours greater than 0, sorted by total logged hours descending.
- **Relational Algebra Expression:**
  1. *Filter high-salary employees (WHERE):*
  $$E_{\text{high}} \leftarrow \sigma_{\text{Salary} \geq 15000000}(\text{EMPLOYEE})$$
  2. *Join with department and task assignment tables (INNER JOIN):*
  $$J_1 \leftarrow (E_{\text{high}} \bowtie_{\text{DeptID}} \text{DEPARTMENT}) \bowtie_{\text{EmpID}} \text{WORKS_ON_TASK}$$
  3. *Group by employee and aggregate task count & logged hours (GROUP BY):*
  $$G_1 \leftarrow {}_{\text{EmpID, FirstName, LastName, DeptName, Salary}}\mathcal{F}_{\text{COUNT(TaskID)}\rightarrow\text{TotalTasks, SUM(HoursLogged)}\rightarrow\text{TotalHoursLogged}}(J_1)$$
  4. *Keep only employees with positive logged hours (HAVING):*
  $$H_1 \leftarrow \sigma_{\text{TotalHoursLogged}>0}(G_1)$$
  5. *Project output columns and sort descending (SELECT, ORDER BY):*
  $$\text{Result}_{5.1} \leftarrow \tau_{\text{TotalHoursLogged}\downarrow}\left(\pi_{\text{EmpID, FullName, DeptName, Salary, TotalTasks, TotalHoursLogged}}(H_1)\right)$$
- **SQL Code:**

```
SELECT
    e.EmpID,
    CONCAT(e.FirstName, ' ', e.LastName) AS FullName,
    d.DeptName,
    e.Salary,
    COUNT(w.TaskID) AS TotalTasks,
    SUM(w.HoursLogged) AS TotalHoursLogged
FROM EMPLOYEE e
INNER JOIN DEPARTMENT d ON e.DeptID = d.DeptID
INNER JOIN WORKS_ON_TASK w ON e.EmpID = w.EmpID
WHERE e.Salary >= 15000000
GROUP BY e.EmpID, e.FirstName, e.LastName, d.DeptName, e.Salary
HAVING TotalHoursLogged > 0
ORDER BY TotalHoursLogged DESC;
```

- **Execution Result:**

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

EmpID	FullName	DeptName	Salary	TotalTasks	TotalHoursLogged
EMP010	Bao Dinh	Site Operations & Construction	22000000.00	2	16.50
EMP008	Nam Vu	Site Operations & Construction	25000000.00	2	11.50
EMP011	Khang Ngo	Site Operations & Construction	20000000.00	1	8.00
EMP009	Son Do	Project Engineering	24000000.00	1	7.50

Query 5.2: Project & Job Site Count by Client

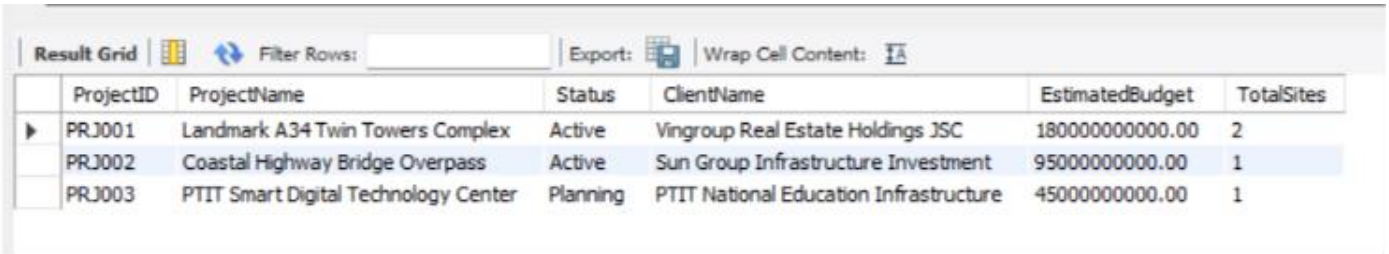
Aggregates non-cancelled high-budget projects by client and counts their active job sites.

- **Problem Statement:** Retrieve all non-cancelled projects with EstimatedBudget > 30,000,000,000 VND, together with the client name and total number of job sites. Keep only projects that have at least 1 job site, sorted by EstimatedBudget descending.
- **Relational Algebra Expression:**
  1. *Filter non-cancelled projects with budget over 30 billion (WHERE):*
  $$P_{\text{high}} \leftarrow \sigma_{\text{Status}\neq\text{'Cancelled'}\wedge\text{EstimatedBudget}>30000000000}(\text{PROJECT})$$
  2. *Join with client and project site tables (INNER JOIN):*
  $$J_2 \leftarrow (P_{\text{high}} \bowtie_{\text{ClientID}} \text{CLIENT}) \bowtie_{\text{ProjectID}} \text{PROJECT_SITE}$$
  3. *Group by project and count job sites (GROUP BY):*
  $$G_2 \leftarrow {}_{\text{ProjectID, ProjectName, Status, ClientName, EstimatedBudget}}\mathcal{F}_{\text{COUNT(SiteID)}\rightarrow\text{TotalSites}}(J_2)$$
  4. *Keep only projects with at least 1 site (HAVING):*
  $$H_2 \leftarrow \sigma_{\text{TotalSites}\geq 1}(G_2)$$
  5. *Project output columns and sort by budget descending (SELECT, ORDER BY):*
  $$\text{Result}_{5.2} \leftarrow \tau_{\text{EstimatedBudget}\downarrow}\left(\pi_{\text{ProjectID, ProjectName, Status, ClientName, EstimatedBudget, TotalSites}}(H_2)\right)$$

- **SQL Code:**

```
SELECT
    p.ProjectID,
    p.ProjectName,
    p.Status,
    c.ClientName,
    p.EstimatedBudget,
    COUNT(ps.SiteID) AS TotalSites
FROM PROJECT p
INNER JOIN CLIENT c ON p.ClientID = c.ClientID
INNER JOIN PROJECT_SITE ps ON p.ProjectID = ps.ProjectID
WHERE p.Status <> 'Cancelled'
  AND p.EstimatedBudget > 30000000000
GROUP BY p.ProjectID, p.ProjectName, p.Status, c.ClientName, p.EstimatedBudget
HAVING TotalSites >= 1
ORDER BY p.EstimatedBudget DESC;
```

- **Execution Result:**



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProjectID	ProjectName	Status	ClientName	EstimatedBudget	TotalSites
PRJ001	Landmark A34 Twin Towers Complex	Active	Vingroup Real Estate Holdings JSC	180000000000.00	2
PRJ002	Coastal Highway Bridge Overpass	Active	Sun Group Infrastructure Investment	95000000000.00	1
PRJ003	PTIT Smart Digital Technology Center	Planning	PTIT National Education Infrastructure	45000000000.00	1

4. CONCLUSION

In Phase 3, we checked the A34Build schema against 1NF, 2NF, and 3NF, removing the partial dependency in EQUIPMENT_DEPLOYMENT and the transitive dependency in DELIVERY_DISPATCH, and implemented the database in MySQL. The 29 core tables, 8 business-rule triggers,sample data, and analytical queries all ran as expected.